



Data Warehouses

Lecture 8



Lecture Goals

Goal:

- Defining tables in MD

- Fact tables

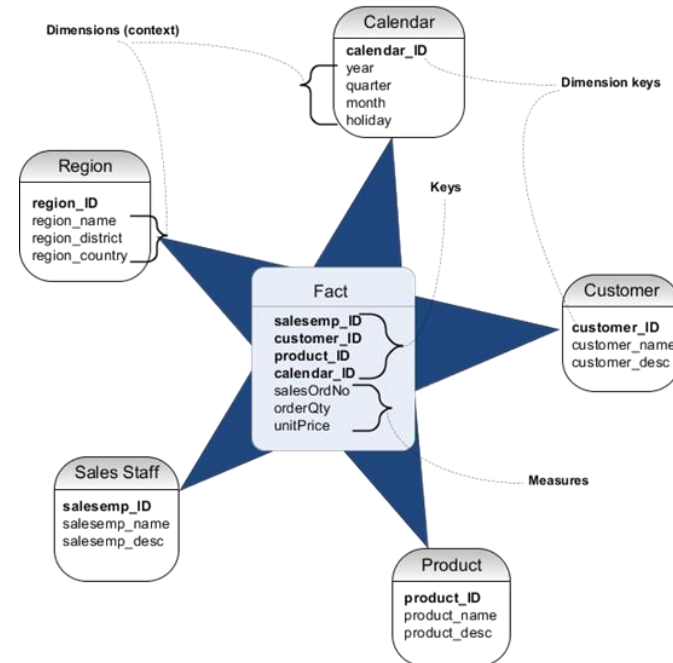
 - Types*

- Dimension tables

 - Hierarchies and types*

- Some notes on measures and dimensions

Facts	Dimensions
Order Dollars	Product
Cost Dollars	Product Description
Quantity Ordered	SKU
	Brand Code
	Brand
	Brand Manager
	Category Code
	Category
	Order Date
	Month of Order
	Quarter of Order
	Fiscal Period of Order
	Year of Order
	Salesperson
	Salesperson ID
	Territory
	Territory Code
	Territory Manager
	Region
	Region Code
	Region VP
	Customer
	Customer ID
	Headquarters State of Customer
	Billing Address
	Billing City
	Billing State
	Billing Zip
	SIC Code of Customer
	Industry Name of Customer



Defining Tables

Dimensions and their values add meaning in many ways:

They accompany facts to provide context on reports.

They are used to filter queries or reports.

They are used to control the scope of aggregation for facts.

They are used to order or sort information.

They are used to define master–detail organization, grouping, subtotalling, and summarization.

A dimensional design

organizes facts and dimensions for storage in a database.

It is common for a set of dimension values to share relationships to one another

independent of facts

these are grouped together in a single table to reflect their natural clustering.

Similarly, measures that are available at the same level of detail are grouped together

facts

Natural affinities exist among the dimension values.

Some of them clearly belong together.

Product and product description both seem to be associated with SKU.

Salesperson name is determined by salesperson ID.

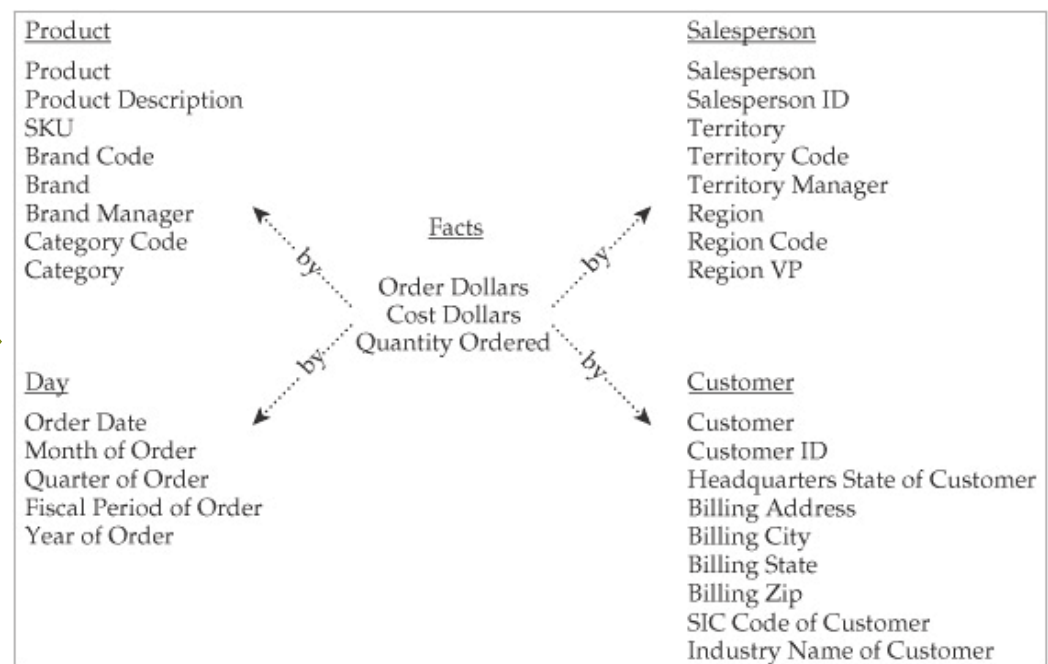
Other dimensions are more distantly associated.

Products, for example, have brands.

Salespeople work in regions.

Facts	Dimensions
Order Dollars	Product
Cost Dollars	Product Description
Quantity Ordered	SKU
	Brand Code
	Brand
	Brand Manager
	Category Code
	Category
	Order Date
	Month of Order
	Quarter of Order
	Fiscal Period of Order
	Year of Order
	Salesperson
	Salesperson ID
	Territory
	Territory Code
	Territory Manager
	Region
	Region Code
	Region VP
	Customer
	Customer ID
	Headquarters State of Customer
	Billing Address
	Billing City
	Billing State
	Billing Zip
	SIC Code of Customer
	Industry Name of Customer

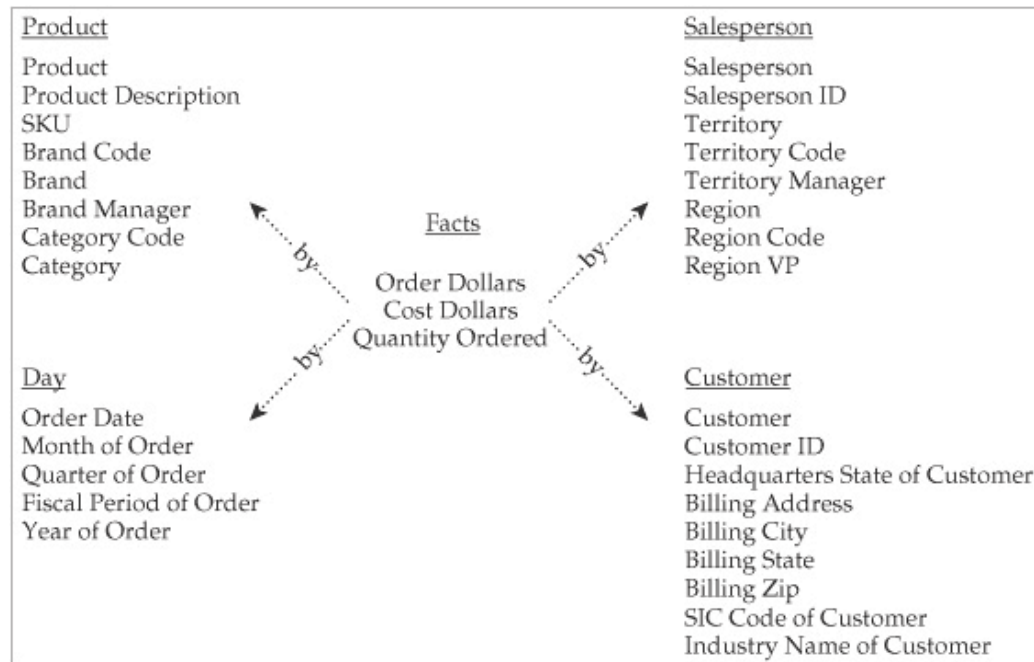
Facts	Dimensions
Order Dollars	Product
Cost Dollars	Product Description
Quantity Ordered	SKU
	Brand Code
	Brand
	Brand Manager
	Category Code
	Category
	Order Date
	Month of Order
	Quarter of Order
	Fiscal Period of Order
	Year of Order
	Salesperson
	Salesperson ID
	Territory
	Territory Code
	Territory Manager
	Region
	Region Code
	Region VP
	Customer
	Customer ID
	Headquarters State of Customer
	Billing Address
	Billing City
	Billing State
	Billing Zip
	SIC Code of Customer
	Industry Name of Customer



The diagram can be easily understood by a businessperson.

“Things I measure” go in the middle;

“the ways I look at them” go in the surrounding boxes, loosely grouped or categorized.



The dimension tables in a star schema are groupings of dimensional attributes that represent major categories of reference information.

These categories usually have strong business significance.

The attributes within a dimension table may bear various relationships to one another

design does not attempt to model these relationships

products, brands, and categories are grouped in a single product table;

salespeople, territories, and regions are grouped in a single salesrep table.

Schema designers sometimes find it challenging to determine the best way to group dimension attributes into tables.

The appropriate categories may not be self-evident.

Two Ways of Relating Dimension Attributes

In a star schema, the relationship between a given pair of dimension attributes may be expressed explicitly or implicitly.

explicit

take the form of joins that intersect in a fact table, which provides an important context for the relationship.

The dimensions may be related in other contexts as well, as represented by other fact tables.

Explicit relationships between dimension rows may be numerous and volatile.

implicit

occur when two attributes are located in the same table

imply a natural affinity between attributes, rather than a relationship that can take many contexts

tend to be more consistent, and they are browseable

Explicit

Every fact table bears foreign key references to dimension tables

references provide the dimensional context for the facts

provide information about relationships between the dimension tables

business process measured by the fact table is the context for this relationship.

These relationships are made explicit by recording additional rows in the fact table

using the appropriate foreign keys

Example

customer <> salesperson

Implicit

Unlike an entity-relationship model, a dimensional model also includes relationships that are not made explicit through joins

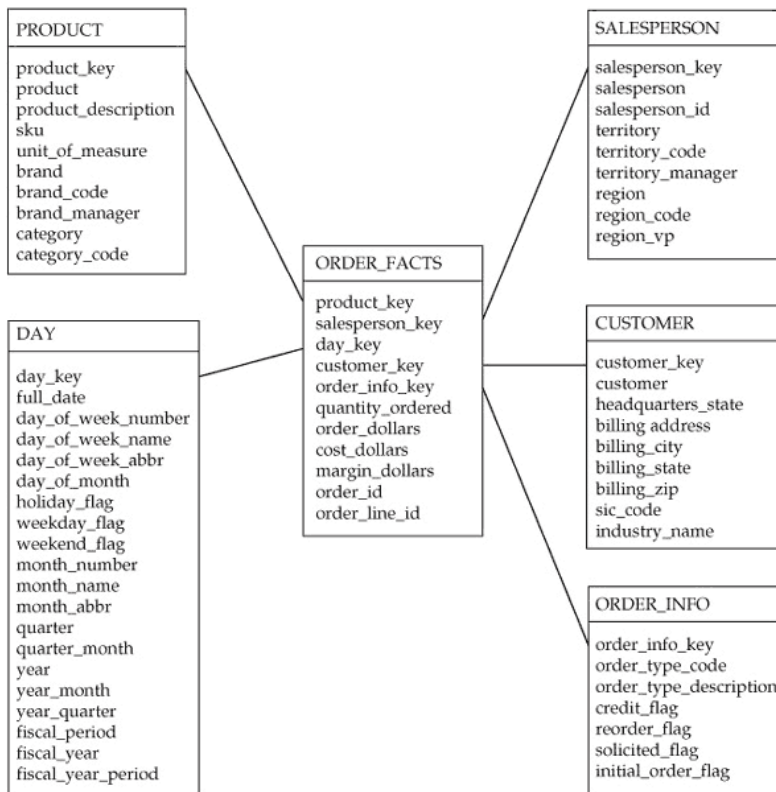
Relationships between dimension attributes can be implied through their coexistence in a table.

tend to exist only in a single context, does not take on multiple contexts
representing a natural affinity rather than one based on process activities

Relationships among attributes in a dimension table may change over time but tend to be less volatile than those of the explicit variety.

Example

product <> brand



Let us use this example

Focus on analysis of orders – as the main business process

Why these dimensions?

why not to merge customer and salesperson?

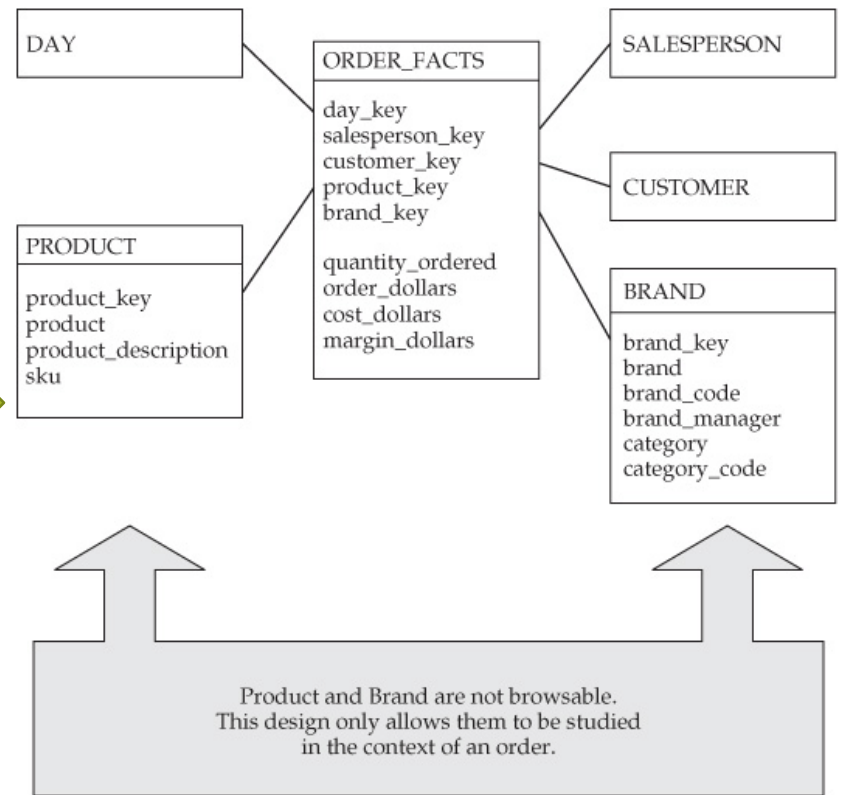
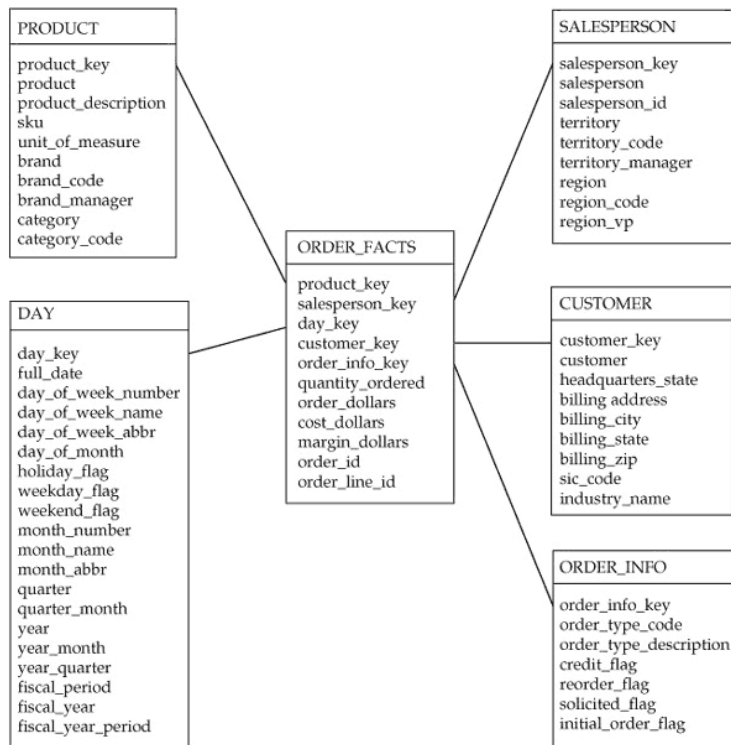
why not distinguish brand as a new dimension?

why not distinguish region as a new dimension?

why there is order info dimension?

why there is order id attribute in dimension?

How to design this structure?



Products and brands are related to one another prior to an order being placed

- transaction is not required to establish this relationship

 - a product has a brand even if there are no orders for it*

- these attributes can only be related in one way, or one context.

Salespeople and customers are brought together only when transactions, such as orders, occur.

- only related based on transactions or activities

 - their relationships will be captured in the fact table*

- allows customers and salespeople to have numerous interactions

 - defined by different transactions*

Product dimension with attributes product and brand

More than one product may share the same brand,

ER model would isolate these attributes in separate tables - via a primary key / foreign key relationship.

Dimensional models are not intended for an operational profile

There is no need to separate brand from product.

Relationship between a product and brand does not take on multiple contexts

Products and brands are related in only one way:

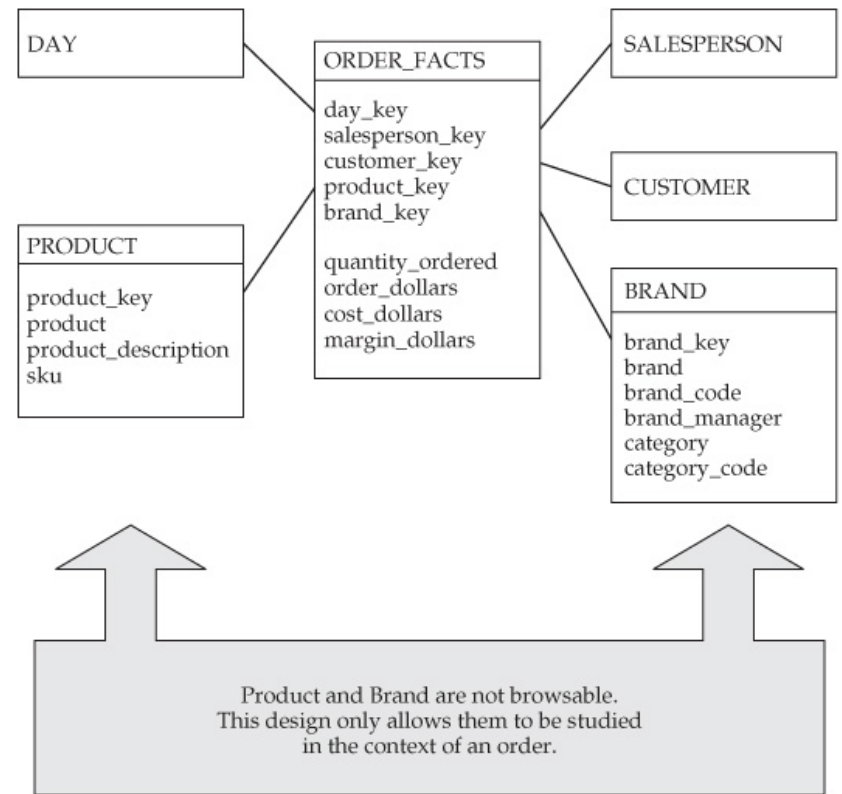
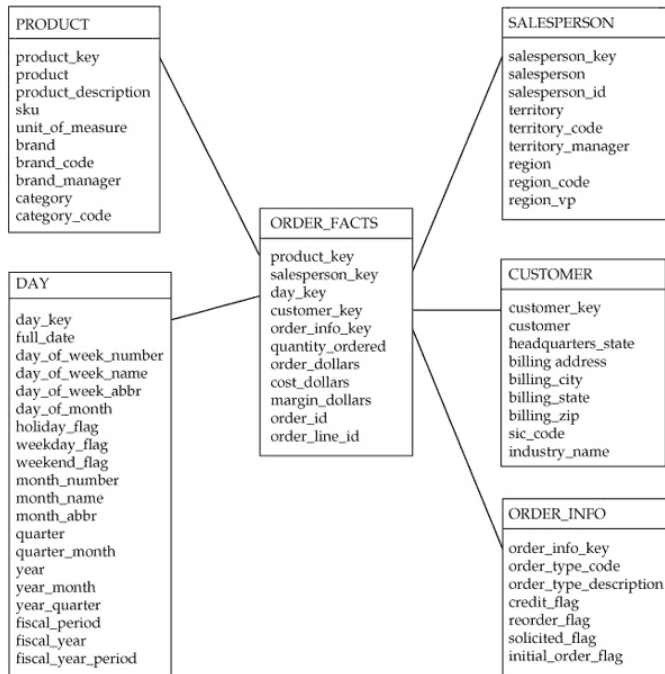
membership in a brand.

It is a natural affinity that does not depend on the execution of business activities.

At a given point in time, a particular product has one associated brand.

This relationship does not depend on a sale, the manufacturing process, or other significant processes tracked by the business.

Unlike the relationship between a customer and salesperson



When Struggling with Dimension Groupings

It is best to look at the ways in which the attributes relate and the ways in which they are used.

For a given pair of attributes, consider the context of the relationship.

Do they share a natural affinity, or can they be related in different contexts?

those that tend to share a stable affinity may be stored together;

those that tend to be related only in the context of events, transactions, or conditions belong in separate fact tables.

When in doubt, you can consider the browsability of alternative designs.

Browsability test

considers how they will be used

a query focused on the values within a dimension is called a browse query – hence browsability test

Consider

Product – Brand

Customer – Salesperson

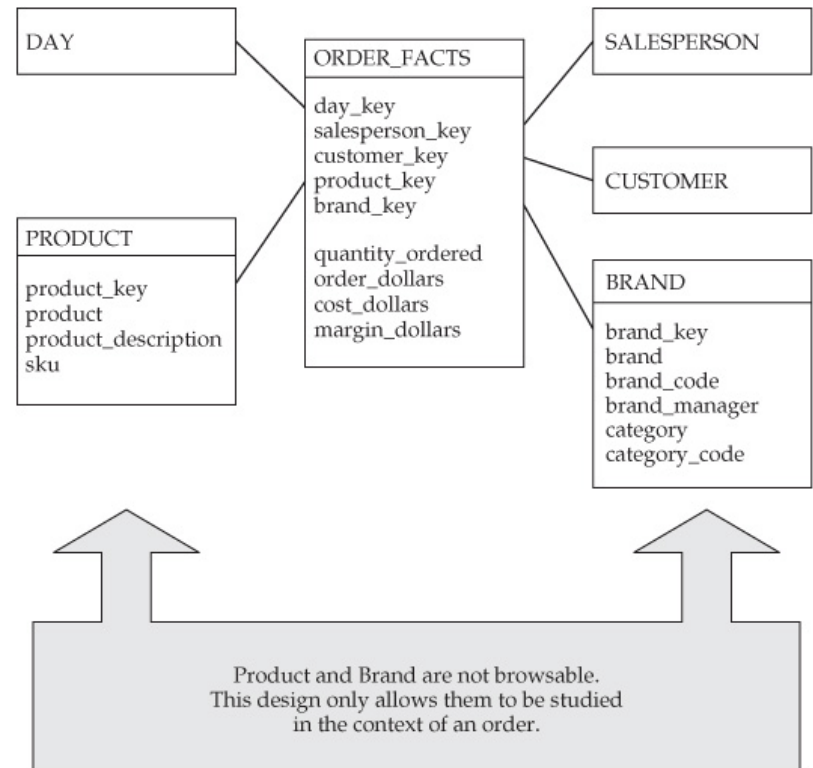
what if a customer is assigned to a salesperson

Browsability Test

Natural affinities can be separated from transactional relationships by evaluating browsability.

Would someone want to browse the values of these attributes together?

Placing them in separate dimensions would prevent this.



Browsability Test

Product and brand

separation of product and brand destroys the browsability of these attributes.

it is only possible to study the intersection of products and brands in the context of orders.

If there is no order for a particular product, it will not be possible to identify its brand.

It makes more sense to place these attributes in a single table.

Salespeople and customers

relationship only when transactions occur

This relationship may also be more volatile.

A given customer may speak with a different salesperson each time an order is placed.

In this case, the transaction defines the relationship and is embodied in the fact table.

Browsability test

In a situation where a business assigns salespeople to specific customers

it may be possible to merge their attributes.

This would allow users to browse the list of customers assigned to a particular salesperson;

however, salespeople and customers may engage in other activities with various contexts, as noted earlier.

Salespeople take calls from customers, process returns, and so forth.

Clearly, the business might consider salespeople and customers to be two separate things.

If it is necessary to track the assignment of customers to salespeople, a factless fact table may be called for.

Assigning attributes to dimensions:

The type of relationship between two attributes is a determining factor:

If a relationship depends on context

this leads to the use of fact tables to capture that relationship -relationship between student number and course number

If a relationship is one of natural affinity

then the attributes are placed in the same dimension -department code, course number

When to distinguish two dimensions

in short:

When two dimensional attributes share a natural affinity, and are only related in one context, they belong in the same dimension table.

When their relationships are determined by transactions or activities, and they can occur in multiple contexts, they should be placed in separate dimension tables.



Note on dimensions

Breaking Up Large Dimensions

A large set of dimension attributes enables the rich analytic capability that makes the data warehouse valuable.

contribute to the formulation of powerful queries and the development of useful reports.

It is not uncommon for dimension tables to contain well over 100 attributes.

Not every dimension is this wide, but every business tends to have two or three major dimensions for which a great deal of information is collected.

Sometimes, a dimension table becomes so wide that database administrators become concerned about its effect on the database.

Such a concern may be purely technical but is completely valid.

Large dimensions can also become a concern for ETL developers.

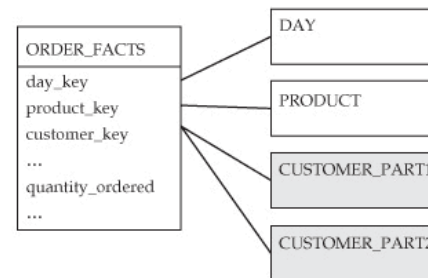
One common solution is an arbitrary separation

simple separation of attributes into two tables

Resultant tables

use the same surrogate key values

share a one-to-one relationship with one another



CUSTOMER_PART1

customer_ key	customer_ id	customer_ name	address_line1
102	A501	Halfway Inc.	192 Elm St.
281	A472	Wooly Links LTD	4710 Maple Ave.
966	A472	Wooly Links LTD	4710 Maple Ave.
1407	A593	ABC Paper	4022 Davis Highway

CUSTOMER_PART2

customer_ key	customer_ id	hq_ location	annual_ revenue
102	A501	Grayville, MT	500,000_1,000,000
281	A472	Springfield, NH	Greater than 1,000,000
966	A472	Lawson, NH	Greater than 1,000,000
1407	A593	North Palte, IA	Less than 500,000

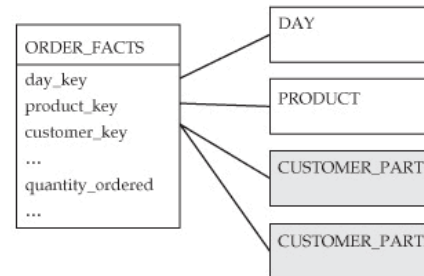
Drawbacks

ETL process is more complicated

star join becomes complex

BI tools might not handle this correctly

performance



CUSTOMER_PART1

customer_key	customer_id	customer_name	address_line1
102	A501	Halfway Inc.	192 Elm St.
281	A472	Wooly Links LTD	4710 Maple Ave.
966	A472	Wooly Links LTD	4710 Maple Ave.
1407	A593	ABC Paper	4022 Davis Highway

CUSTOMER_PART2

customer_key	customer_id	hq_location	annual_revenue	
102	A501	Grayville, MT	500,000_1,000,000	
281	A472	Springfield, NH	Greater than 1,000,000	
966	A472	Lawson, NH	Greater than 1,000,000	
1407	A593	North Palte, IA	Less than 500,000	

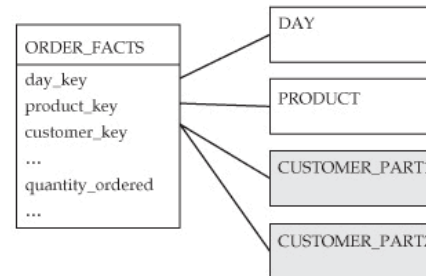
Alternatives

Maybe two dimensions?

Move long text fields to an
outrigger

Identify subtypes

mini-dimension



CUSTOMER_PART1

customer_ key	customer_ id	customer_ name	address_line1
102	A501	Halfway Inc.	192 Elm St.
281	A472	Wooly Links LTD	4710 Maple Ave.
966	A472	Wooly Links LTD	4710 Maple Ave.
1407	A593	ABC Paper	4022 Davis Highway

CUSTOMER_PART2

customer_ key	customer_ id	hq_ location	annual_ revenue	
102	A501	Grayville, MT	500,000_1,000,000	
281	A472	Springfield, NH	Greater than 1,000,000	
966	A472	Lawson, NH	Greater than 1,000,000	
1407	A593	North Palte, IA	Less than 500,000	

Two Dimensions

often a sign that there are two distinct dimensions

dimension can be redesigned as two tables, each with its own surrogate key

Relocating Free-Form Text Fields to an Outtrigger

Excessive row length is often a result of the inclusion of several free-form text fields.

may contain unstructured data that is occasionally used to filter reports

they can be relocated to a separate table and replaced with a foreign key reference – example of outrigger technique

Looking for Subtypes

In many cases, a dimension will contain large groups of attributes that each apply to only one subset of its rows.

In an operational system, this is referred to as subtyping.

Example,

the attributes of a dimension representing a product may vary significantly, depending on what type of product it is.

Books, magazines and compact discs share some common attributes, but each subtype also has its own set of attributes.

Building a core dimension with only the shared attributes, and separate custom dimensions for each subtype.

Dimensions

Dimensions can be added to queries in different combinations to answer a wide variety of questions.

The larger the set of dimension attributes, the more ways that facts can be analyzed.

Dimension tables with a large number of attributes maximize analytic value.

They can be thought of as wide.

In addition to storing common attributes, dimension tables store commonly used combinations of attributes.

Codes may be supplemented with corresponding description values.

Flags are translated from Boolean values into descriptive text,

Multi-part fields are both preserved and broken down into constituent pieces.

It is also important not to overlook numeric attributes that can serve as dimensions.

Dimensions

Codes and Descriptions

In operational systems, it is common for the list of appropriate values in a domain to be described using codes.

A separate table is used to provide the corresponding descriptions – reference / lookup values

Descriptions may be more useful than the codes themselves.

Flags and Their Values

Columns whose values are Boolean in nature are usually referred to as flags.

In a dimensional design, these flags may be used to filter queries or group facts.

By storing a descriptive value for the flag, we make using the flag easier.

These descriptors are far more useful than 0/1 or Y/N, and can also be used less ambiguously when defining a query predicate or filter.

Multiple-Part Columns

Operational systems often contain attributes that have multiple parts, each part bearing some sort of significance.

In a dimensional design, the entire attribute may be stored, along with additional attributes that isolate its constituent parts.

If these subcomponents are codes, they may also be accompanied by corresponding description values.

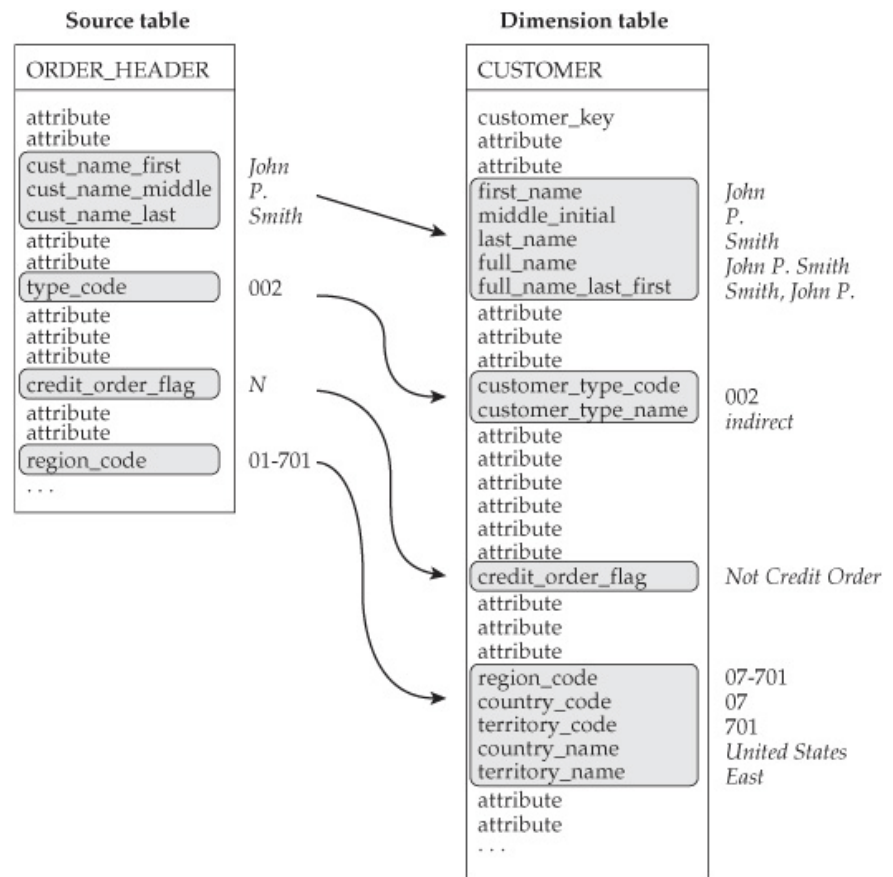
For example, the operational system records a region code in the format XX-YYY. The first part of this code designates a country, and the second part designates a territory within that country..

Dimensions

In operational systems, it is common practice to break data elements down to constituent parts whenever possible.

These components have analytic value and, of course, will be included in a dimensional design.

Unlike the operational schema, however, the dimensional schema should also include dimensions that represent common combinations of these elements.



Dimensions

Dimensions with Numeric Values

While the majority of dimensions contain data that is textual, sometimes dimensions contain numeric data.

For example, sizes, telephone numbers, and zip codes.

All of these examples are clearly dimensions.

They will be used to provide context for facts, to order data, to control aggregation, or to filter query results.

Some numeric attributes are less easy to identify as dimensions.

For example, the unit price associated with an order is numeric.

If 100 widgets are sold at \$10 apiece

is the \$10 unit price a fact or a dimension?

Dimensions

Dimensions with Numeric Values

It is not always clear whether a numeric data element is a fact or a dimension.
pay close attention to how it will be used.

If the element values are used to filter queries, order data, control aggregation, or drive master–detail relationships, it is most likely a dimension.

In general, if an attribute is commonly aggregated or summarized, it is a fact.

If it is used to drive aggregations or summarizations, however, it is a dimension.

In the case of a unit price, it is not useful to sum unit prices across multiple orders. On the other hand, it is useful to group orders by unit price, perhaps to answer the question, “How many did I sell at \$10 each versus \$12 each?” The unit price is, therefore, behaving as a dimension.

Dimensional Modelling

Consider the following issues:

Dimension Identification

An attribute as a dimension – multiple queries concerning this aspect – performance

Ex. Color of a product; Color aspects can form a separate dimension

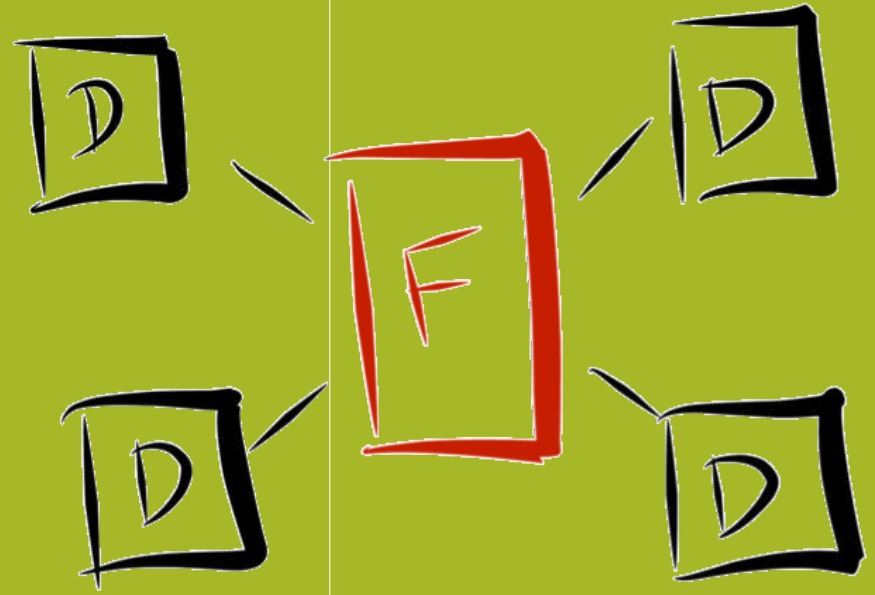
Attribute Determination

Relational stores relations between entites – here we can discretise them – performance

Ex. Medical doctor – specialisation; as specialisation1, specialisation2, ...

Access information

Additional information about user interested in a particular element



Fact Table

Fact Tables

Note:

one process can be broken down into several subprocesses.

e.g., sales process may be broken down into subprocesses for order entry, shipment, invoicing, and returns management.

We can run into a complication: sales seems to be a process, but it also seems to be made up of other processes.

Does the study of sales require multiple fact tables, or just one?

Consider:

Do these facts occur simultaneously?

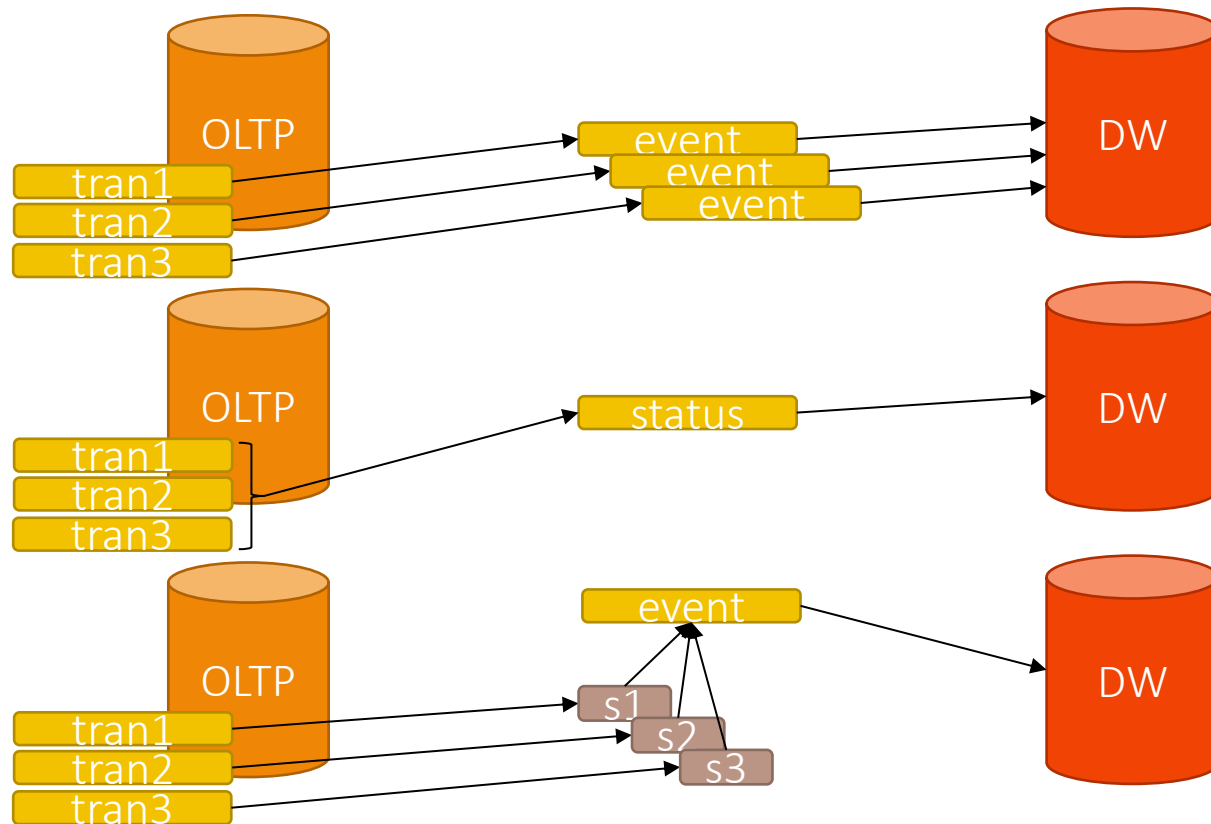
How to interpret „simultaneous“?

Are these facts available at the same level of detail (or grain)?

How to interpret „available“?

If the answer to either of these questions is “no,” the facts should represent different processes.

Fact Tables



Fact Tables

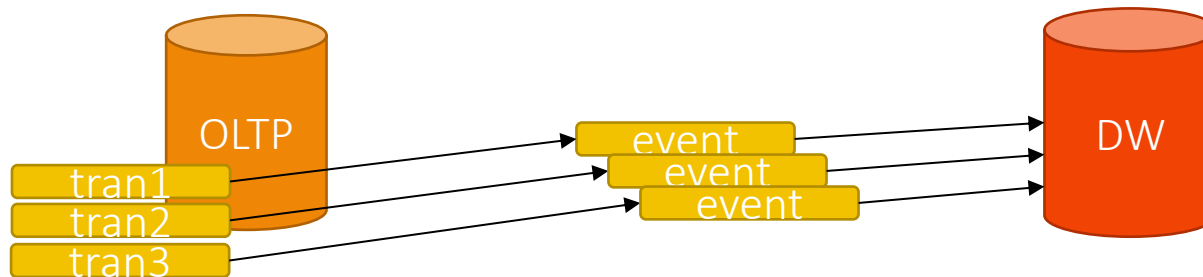
Types of facts - transaction fact

corresponds to a measurement of performance of an event at a particular point in multidimensional space

grain is typically adequate to the transaction

Example

order line



Fact Tables

Account	Transaction Date	Transaction Type	Transaction Amount
12345	3/1/2017	Initial Deposit	\$1,000
12345	3/3/2017	Payroll Deposit	\$500
12345	3/3/2017	ATM Withdrawal	\$100
12345	3/7/2017	Check	\$75
12345	3/10/2017	Payroll Deposit	\$500
12345	3/11/2017	ATM	\$250
12345	3/17/2017	Payroll Deposit	\$500
12345	3/20/2017	Check	\$110
12345	3/22/2017	Web Payment	\$135
12345	3/24/2017	Payroll Deposit	\$500
12345	3/24/2017	Web Payment	\$90
12345	3/24/2017	ATM Withdrawal	\$125
12345	3/28/2017	Check	\$50
12345	3/30/2017	ATM Withdrawal	\$65
12345	3/31/2017	Payroll Deposit	\$500

Fact Tables

Types of facts- transaction fact

grain is not always the individual transaction

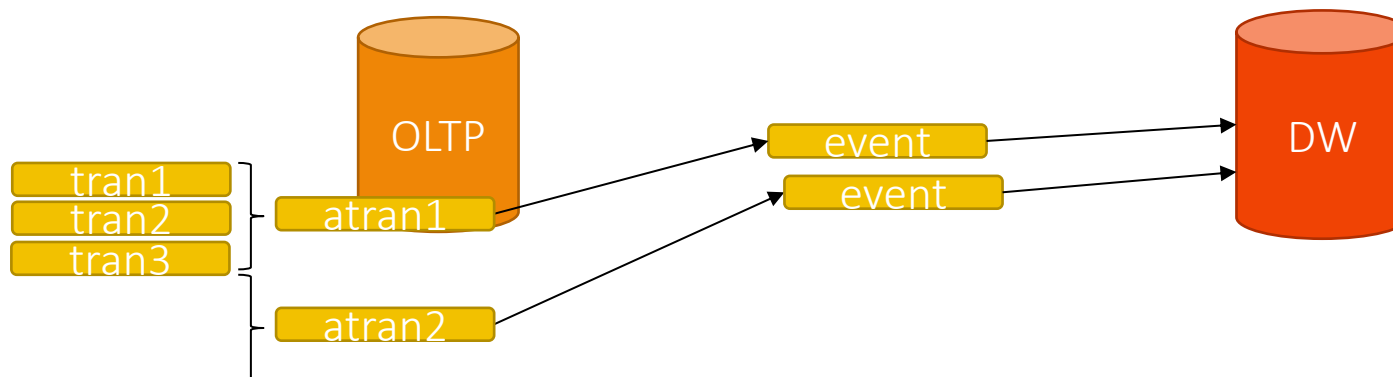
Each row of the fact table describes specific events, though not individual events

many real-world transaction fact tables summarize activities

because detail is available elsewhere or because the transaction volume is too large

Example

aggregated orders by day, salesperson, customer, and product



Fact Tables

Types of facts – periodic snapshot fact

measuring the effect of a series of transactions – called status measurements

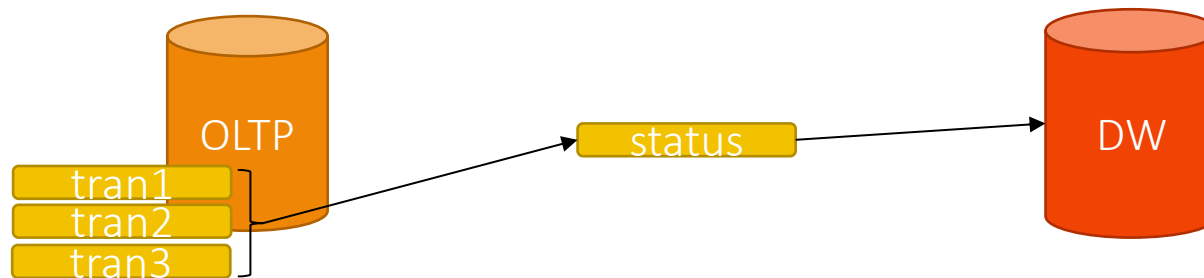
samples the measurement in question at a predetermined interval

status can be sometimes discerned by aggregating the transactions that contribute to it

the grain is the period, not the individual transaction

Examples

account balances and inventory levels.



Fact Tables

Account	Transaction Date	Transaction Type	Transaction Amount
12345	3/1/2017	Initial Deposit	\$1,000
12345	3/3/2017	Payroll Deposit	\$500
12345	3/3/2017	ATM Withdrawal	\$100
12345	3/7/2017	Check	\$75
12345	3/10/2017	Payroll Deposit	\$500
12345	3/11/2017	ATM	\$250
12345	3/17/2017	Payroll Deposit	\$500
12345	3/20/2017	Check	\$110
12345	3/22/2017	Web Payment	\$135
12345	3/24/2017	Payroll Deposit	\$500
12345	3/24/2017	Web Payment	\$90
12345	3/24/2017	ATM Withdrawal	\$125
12345	3/28/2017	Check	\$50
12345	3/30/2017	ATM Withdrawal	\$65
12345	3/31/2017	Payroll Deposit	\$500

Account	Date	Balance
12345	3/1/2017	\$1,000
12345	3/2/2017	\$1,000
12345	3/3/2017	\$1,400
12345	3/4/2017	\$1,400
12345	3/5/2017	\$1,400
12345	3/6/2017	\$1,400
12345	3/7/2017	\$1,325
12345	3/8/2017	\$1,325
12345	3/9/2017	\$1,325
12345	3/10/2017	\$1,825
12345	3/11/2017	\$1,575
12345	3/12/2017	\$1,575
12345	3/13/2017	\$1,575
12345	3/14/2017	\$1,575
12345	3/15/2017	\$1,575
12345	3/16/2017	\$1,575
12345	3/17/2017	\$2,075
12345	3/18/2017	\$2,075
12345	3/19/2017	\$2,075
12345	3/20/2017	\$1,965
12345	3/21/2017	\$1,965
12345	3/22/2017	\$1,830
12345	3/23/2017	\$1,830
12345	3/24/2017	\$2,115
12345	3/25/2017	\$2,115
12345	3/26/2017	\$2,115
12345	3/27/2017	\$2,115
12345	3/28/2017	\$2,065
12345	3/29/2017	\$2,065
12345	3/30/2017	\$2,000
12345	3/31/2017	\$2,500

Fact Tables

Types of facts - periodic snapshot fact

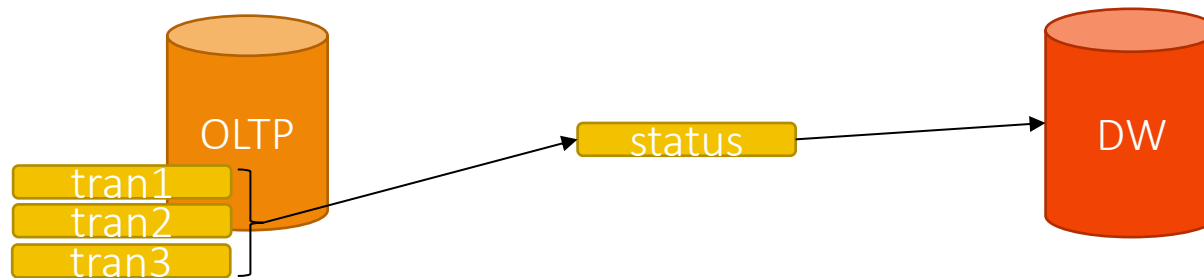
measuring the effect of a series of transactions is as useful as measuring the transactions themselves

Status can often be discerned by aggregating the transactions that contribute to it

Some status measurements cannot be described as the effect of a series of transactions

When the measurement of status is important, a transaction fact table is inefficient at best

The snapshot fact table samples the measurement in question at a predetermined interval.



Fact Tables

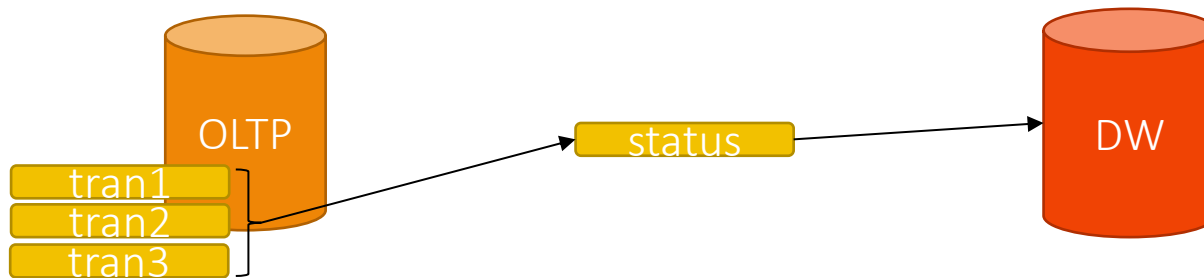
Types of facts - periodic snapshot fact

A snapshot fact table design has several properties

the grain of a snapshot fact table is usually declared in dimensional terms

snapshots are dense

snapshot model will contain at least one fact that exhibits a property known as semi-additivity



Fact Tables

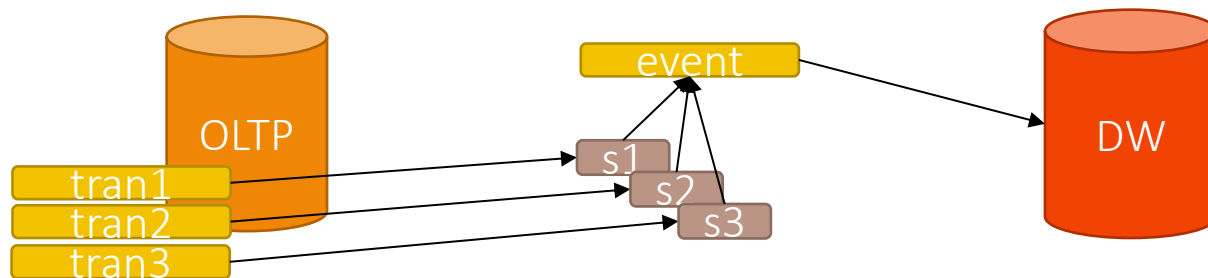
Types of facts - accumulating snapshot fact

Many business processes can be described as a series of stages, steps, or statuses through which something must pass

The efficiency of a process is often measured as the amount of time it takes to complete one or more steps.

Example

In technical support, a problem ticket is logged, assigned, diagnosed, and closed.



Fact Tables

CLAIM KEY	CUSTOMER KEY	POLICY KEY	CLAIM DATE	INVESTIGATION DATE	DAYS TO INVESTIGATION	REVIEW DATE	DAYS TO REVIEW	DESCISION DATE	DAYS TO DECISION	PAYMENT DATE	DAYS TO PAYMENT	PROCESS COMPLETE FLAG
345623	23113	5567889	1/3/2017	5-Jan	2	9-Jan	6	10-Jan	7	13-Jan	10	1
345624	78345	4223131	1/3/2017	6-Jan	3	9-Jan	6	12-Jan	9	17-Jan	14	1
345625	44560	7997675	1/3/2017	9-Jan	6	13-Jan	10	16-Jan	13			1
345626	12123	2342345	1/3/2017	4-Jan	1	11-Jan	8	13-Jan	10	17-Jan	14	1
...	...	...	...	...	...	...	...	...	...	...	...	...
341123	88767	2627433	2/8/2017	9-Feb	1	10-Feb	2	13-Feb	5	14-Feb	6	1
341124	94332	5454626	2/8/2017	9-Feb	1	10-Feb	2	10-Feb	2			0
341125	54565	4356742	2/8/2017	9-Feb	1	13-Feb	5					0
341126	33356	2745649	2/8/2017	13-Feb	5							0

NOTE: The current date is 2/15/2017

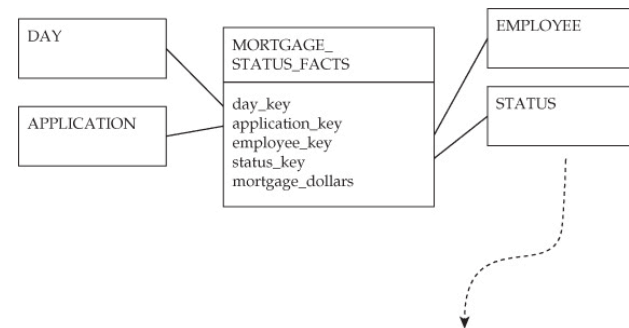
Fact Tables

Types of facts:

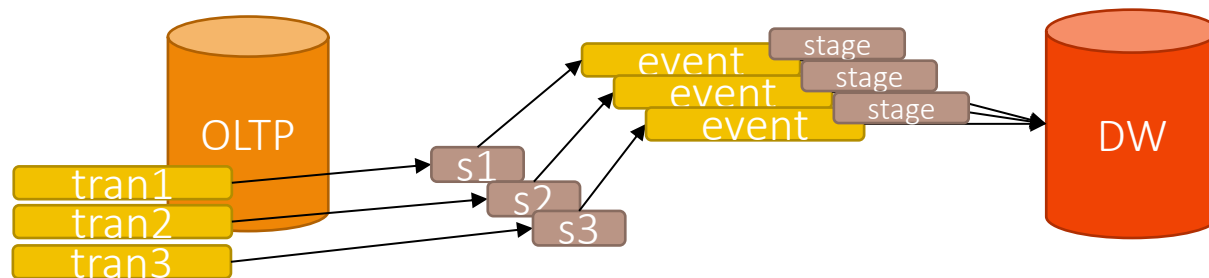
accumulating snapshots in transactional fact table

fact table records one row for each status change or milestone achieved

contents of the status dimension reveals the major processing steps



status_key	status_code	status	status_description
1000	S1	Submitted	Submitted. Under review by mortgage officer.
1001	A2	Reviewed	Reviewed. Documentation being gathered by processor.
1002	P2	Processed	Processed. Under examination by underwriter.
1003	U2	Underwritten	Underwritten. Awaiting settlement.
1004	EX	Settled	Settled.



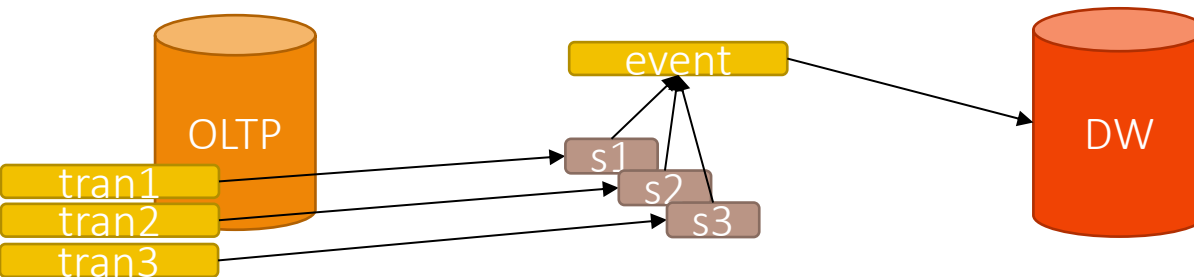
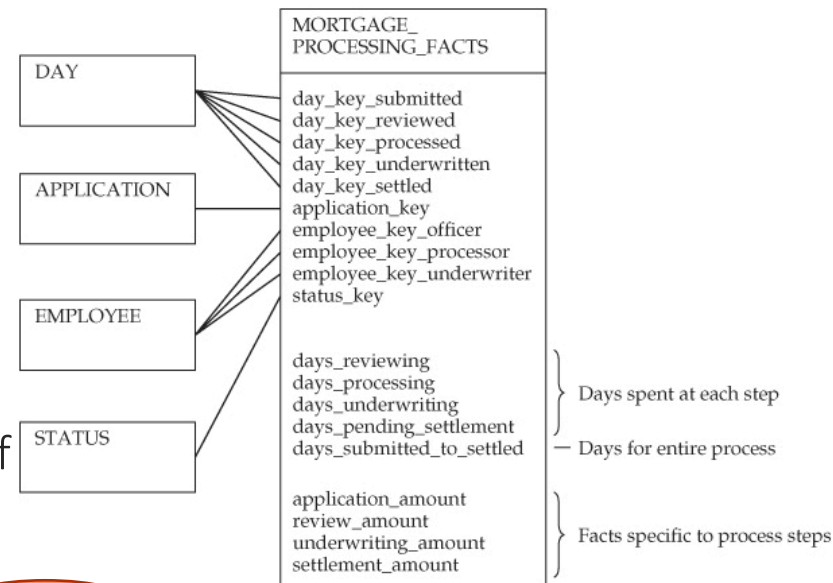
Fact Tables

Types of facts - accumulating snapshot fact

The grain is framed in terms of an identifiable entity that passes through the business process.

The fact table will have exactly one row for each instance of the entity.

Multiple relationships to the day dimension represent the achievement of each significant milestone or status.



Fact Tables

Types of facts - accumulating snapshot fact

On Day 1 (Submitted; under review by officer):

application_ key	day_key_ submitted	day_key_ reviewed	day_key_ processed	day_key_ underwritten	day_key_ closing	application_ amount	review_ amount	underwriting_ amount	days_ reviewing	days_ processing	...
1011	1021	0000	0000	0000	0000	100,000	0	0	0	0	

Day 2 (No status change):

application_ key	day_key_ submitted	day_key_ reviewed	day_key_ processed	day_key_ underwritten	day_key_ closing	application_ amount	review_ amount	underwriting_ amount	days_ reviewing	days_ processing	...
1011	1021	0000	0000	0000	0000	100,000	0	0	1	0	

Days 3–9 (not shown)...

Day 10 (Reviewed; documents being gathered by processor):

application_ key	day_key_ submitted	day_key_ reviewed	day_key_ processed	day_key_ underwritten	day_key_ closing	application_ amount	review_ amount	underwriting_ amount	days_ reviewing	days_ processing	...
1011	1021	1031	0000	0000	0000	100,000	90,000	0	9	0	

Day 11 (No status change):

application_ key	day_key_ submitted	day_key_ reviewed	day_key_ processed	day_key_ underwritten	day_key_ closing	application_ amount	review_ amount	underwriting_ amount	days_ reviewing	days_ processing	...
1011	1021	1031	0000	0000	0000	100,000	90,000	0	9	1	

Remaining steps...

Fact Tables

Types of facts:

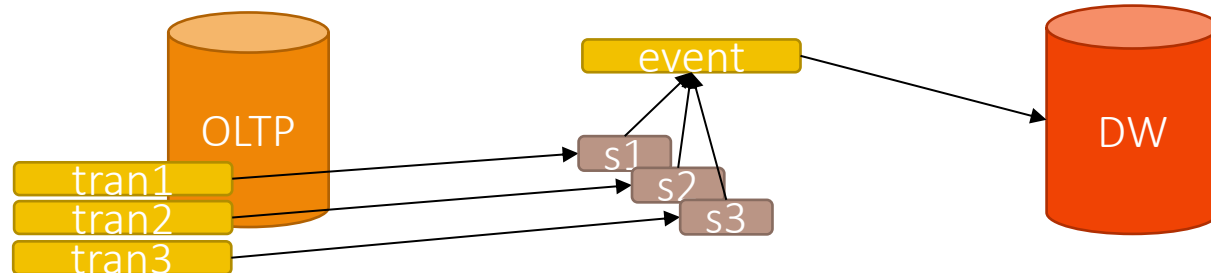
accumulating snapshot fact

summarizes the measurement events occurring at predictable steps between the beginning and the end of a process

contains multiple references to the date dimension

date foreign key in the fact table for each critical milestone in the process

lags - track the number of days (or minutes) spent between each milestone



Fact Tables

Types of Facts

"Fact-less" facts

A fact per event (customer contact)

so-called event fact

No numerical measures

An event has happened for a given dimension value combination

For example, an event of a student attending a class on a given day

may not have a recorded numerical measure associated with it,

but a fact row with foreign keys for day, student, teacher, location, and class

what if a student never attended the class?

Fact Tables

Types of Facts

”Fact-less” facts

Fact per relation

So-called coverage fact

can also be used to analyze what didn't happen

a fact-less coverage table that contains all the possibilities of events that might happen
and an activity table that contains the events that did happen

For example, an event of students assigned to a class in a given semester

may not have a recorded numeric fact,

but a fact row with foreign keys for semester, student and class

allows to analyse the absence rate

Conformed

Conformed Facts

The same measurement might appear in separate fact tables

care must be taken to make sure the technical definitions of the facts are identical if they are to be compared or computed together

If the separate fact definitions are consistent, the conformed facts should be identically named

but if they are incompatible, they should be differently named to alert the business users and BI applications.

Consolidated

Consolidated Fact Tables

It is often convenient to combine facts from multiple processes together into a single consolidated fact table if they can be expressed at the same grain.

For example, sales actuals can be consolidated with sales forecasts in a single fact table to make the task of analyzing actuals versus forecasts simple and fast, as compared to assembling a drill-across application using separate fact tables.

add burden to the ETL processing but ease the analytic burden on the BI applications.

They should be considered for cross-process metrics that are frequently analyzed together.

Nulls

Nulls in Fact Tables

Null-valued measurements typically behave gracefully in fact tables

The aggregate functions (SUM, COUNT, MIN, MAX, and AVG) all do the “right thing” with null facts

Depending on the tools used

Nulls must be avoided in the fact table's foreign keys

these nulls would automatically cause a referential integrity violation

rather than a null foreign key, the associated dimension table must have a default row (and surrogate key) representing the unknown or not applicable condition.

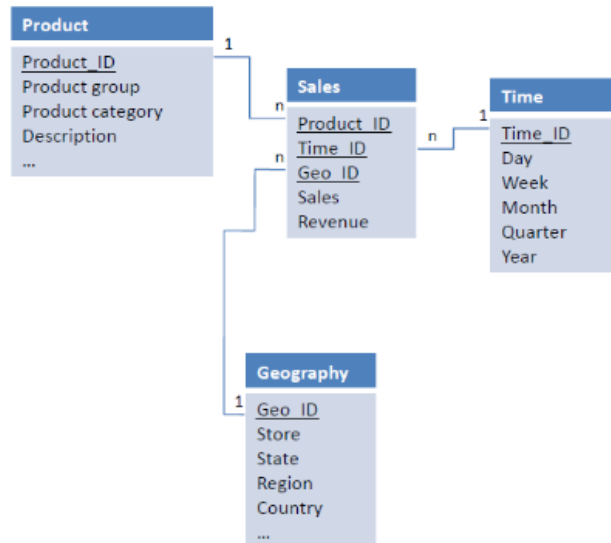
Keys

Fact table keys

Foreign keys

Do we need them?

How should they be formulated?



Fact table keys

Primary keys

Do we need it?

Logical level?

Physical level?

How to represent it?

Surrogate key?

Aggregates

Aggregate fact tables

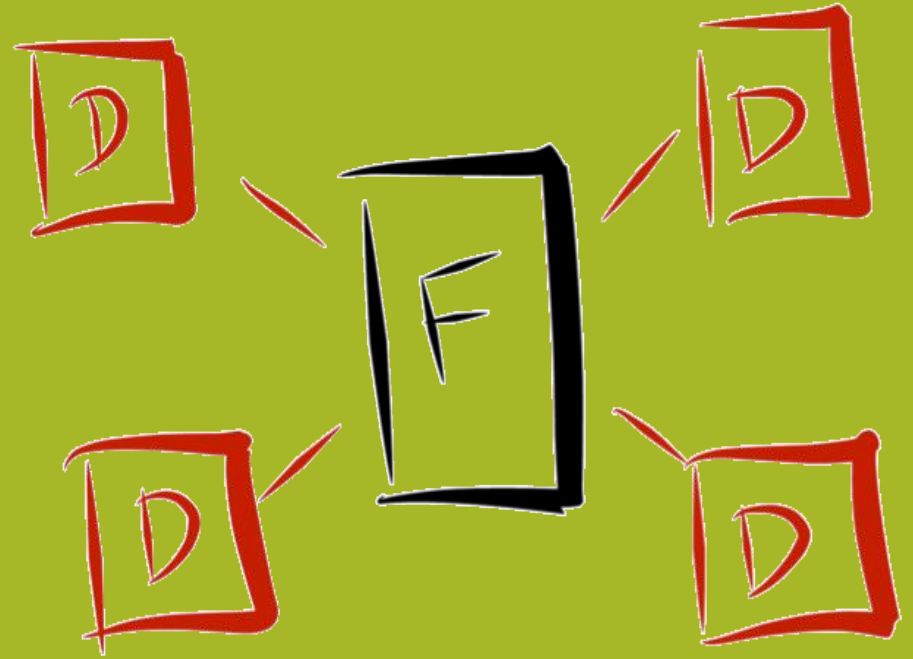
- numeric rollups of atomic fact table data built solely to accelerate query performance

- should be available to the BI layer at the same time as the atomic fact tables so that BI tools smoothly choose the appropriate aggregate level at query time

- This process is known as aggregate navigation*

- A properly designed set of aggregates should behave like database indexes, which accelerate query performance but are not encountered directly by the BI applications or business users.

- contain foreign keys to shrunken conformed dimensions, as well as aggregated facts created by summing measures from more atomic fact tables.



Dimension Table(s)

Dimension Table

Dimension's primary key

The foundation of any star schema design is the foreign keys that exist between the fact and dimension tables

do not use the primary and foreign keys that exist in the source databases

surrogate key

Keys

Natural keys created by operational source systems are subject to business rules outside the control of the DW/BI system

For instance, an employee number (natural key) may be changed if the employee resigns and then is rehired.

When the data warehouse wants to have a single key for that employee, a new durable key must be created that is persistent and does not change in this situation.

This key is sometimes referred to as a durable supernatural key.

The best durable keys have a format that is independent of the original business process and thus should be simple integers assigned in sequence beginning with 1.

While multiple surrogate keys may be associated with an employee over time as their profile changes, the durable key never changes.

Dimension Table

Dimension's primary key

A dimension table is designed with one column serving as a unique primary key

primary key should not be the operational system's natural key

there will be multiple dimension rows for that natural key when changes are tracked over time.

natural keys for a dimension may be created by more than one source system, and these natural keys may be incompatible or poorly administered

rather than using explicit natural keys or natural keys with appended dates, you should create anonymous integer primary keys for every dimension.

These dimension surrogate keys are usually simple integers

assigned in sequence, starting with the value 1, every time a new key is needed.

Dimension Table

Dimension's primary key

The dimension table will have two keys:

a key from the data source (the original OLTP key)

a surrogate key, generated during the ETL process

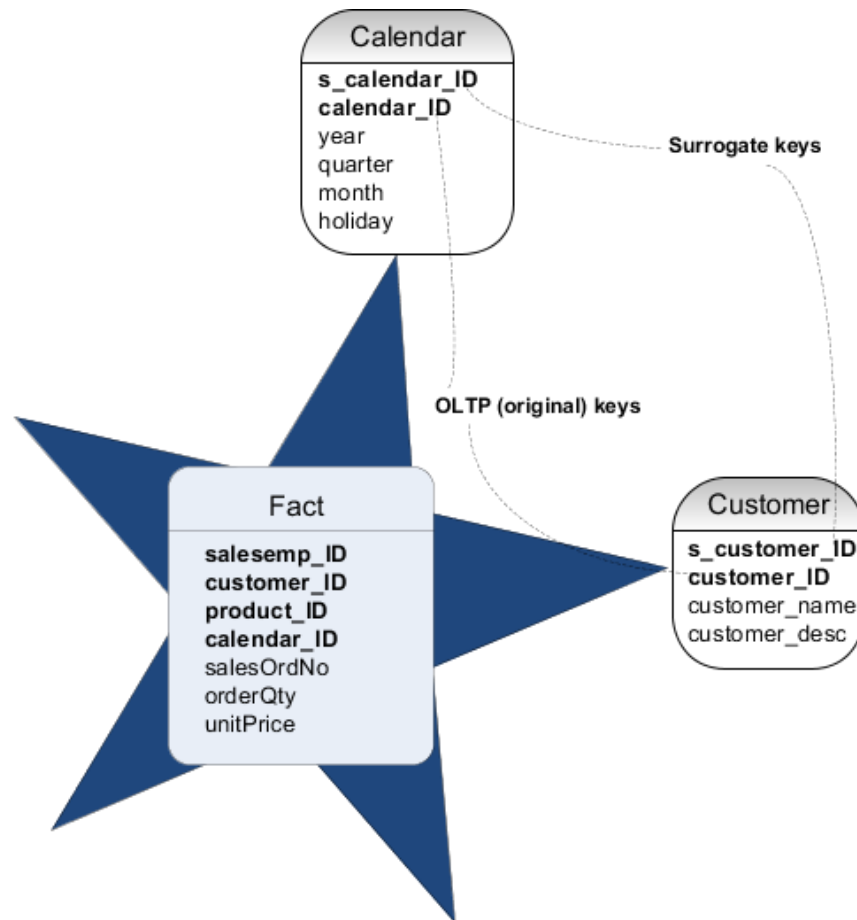
The surrogate key enables you to include additional data sources later without having to consider the possibility of duplicate OLTP key values.

These keys are transparent and easy to structure for reporting tools.

The date dimension is exempt from the surrogate key rule

highly predictable and stable dimension can use a more meaningful primary key

Dimension Table



Drilling

Drilling Down

the most fundamental way data is analyzed by business users

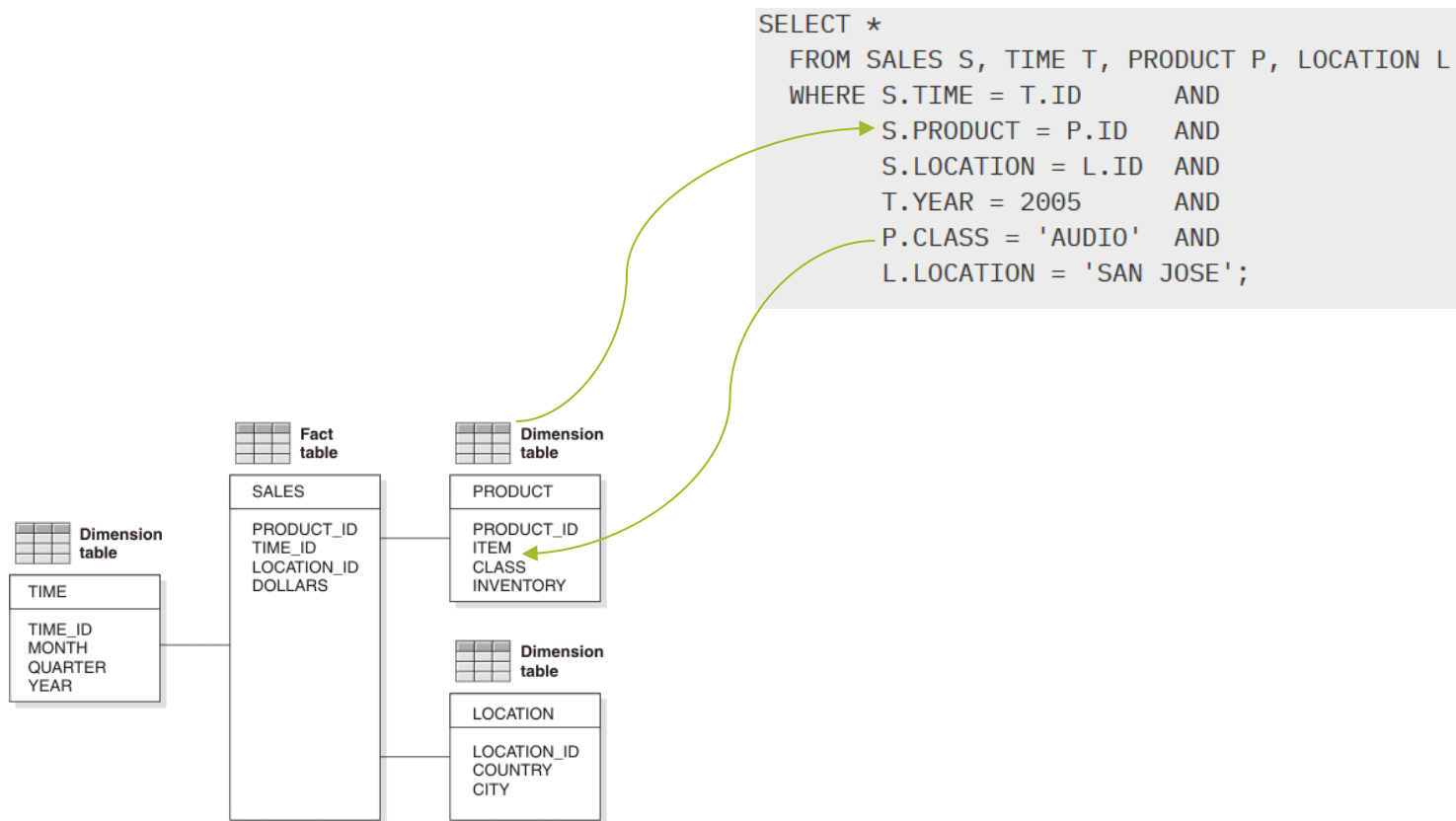
simply means adding a row header to an existing query

the new row header is a dimension attribute appended to the GROUP BY expression in an SQL query

The attribute can come from any dimension attached to the fact table in the query

In general, drilling down does not require the definition of predetermined hierarchies or drill-down paths

Dimension Table



Nulls

Null-valued dimension attributes result when a given dimension row has not been fully populated, or when there are attributes that are not applicable to all the dimension's rows

In both cases, we recommend substituting a descriptive string
such as Unknown or Not Applicable in place of the null value.

Nulls in dimension attributes should be avoided because different databases handle grouping and constraining on nulls inconsistently.

Dimension Table

In addition to storing common attributes, dimension tables store commonly used combinations of attributes.

- Codes may be supplemented with corresponding description values.

- Flags are translated from Boolean values into descriptive text,

- Multi-part fields are both preserved and broken down into constituent pieces.

It is also important not to overlook numeric attributes that can serve as dimensions.

Dimension Table

Codes and Descriptions

In operational systems, it is common for the list of appropriate values in a domain to be described using codes.

A separate table is used to provide the corresponding descriptions – reference / lookup values

Descriptions may be more useful than the codes themselves.

Flags and Their Values

Columns whose values are Boolean in nature are usually referred to as flags.

In a dimensional design, these flags may be used to filter queries or group facts.

By storing a descriptive value for the flag, we make using the flag easier.

These descriptors are far more useful than 0/1 or Y/N, and can also be used less ambiguously when defining a query predicate or filter.

Multiple-Part Columns

Operational systems often contain attributes that have multiple parts, each part bearing some sort of significance.

In a dimensional design, the entire attribute may be stored, along with additional attributes that isolate its constituent parts.

If these subcomponents are codes, they may also be accompanied by corresponding description values.

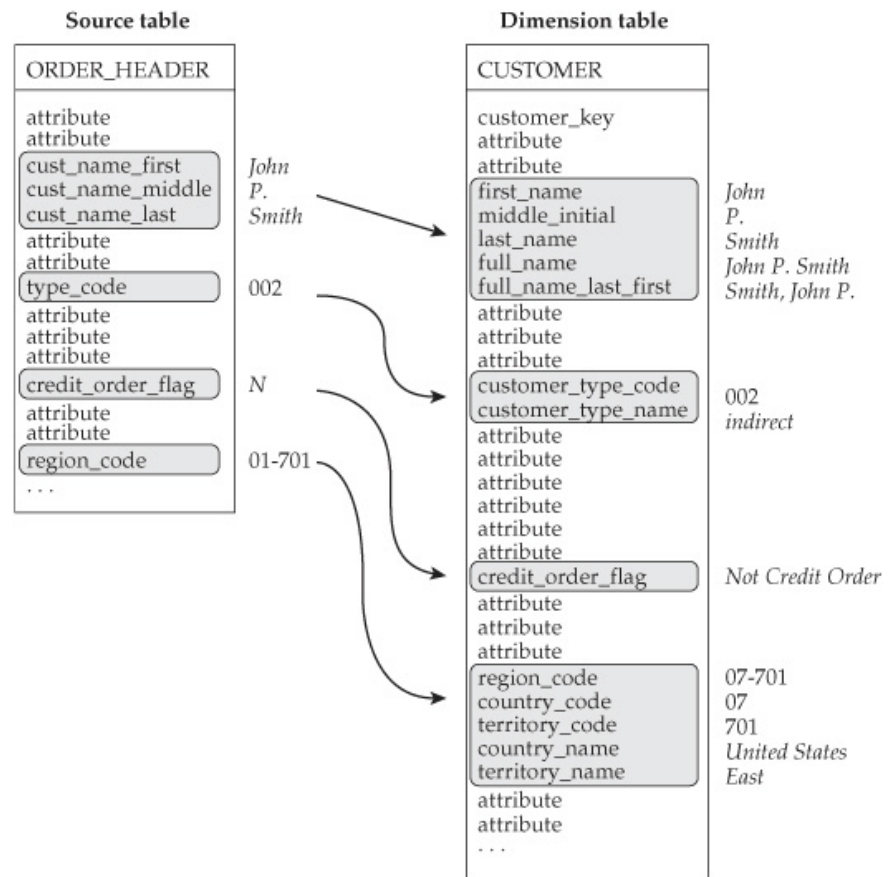
For example, the operational system records a region code in the format XX-YYY. The first part of this code designates a country, and the second part designates a territory within that country..

Dimension Table

In operational systems, it is common practice to break data elements down to constituent parts whenever possible.

These components have analytic value and, of course, will be included in a dimensional design.

Unlike the operational schema, however, the dimensional schema should also include dimensions that represent common combinations of these elements.



Dimension Table

Dimensions with Numeric Values

While most dimensions contain data that is textual, sometimes dimensions contain numeric data.

For example, sizes, telephone numbers, and zip codes.

All of these examples are clearly dimensions.

They will be used to provide context for facts, to order data, to control aggregation, or to filter query results.

Some numeric attributes are less easy to identify as dimensions.

For example, the unit price associated with an order is numeric.

If 100 widgets are sold at \$10 apiece

is the \$10 unit price a fact or a dimension?

Dimension Table

Dimensions with Numeric Values

It is not always clear whether a numeric data element is a fact or a dimension.

pay close attention to how it will be used.

If the element values are used to filter queries, order data, control aggregation, or drive master–detail relationships, it is most likely a dimension.

In general, if an attribute is commonly aggregated or summarized, it is a fact.

If it is used to drive aggregations or summarizations, however, it is a dimension.

In the case of a unit price, it is not useful to sum unit prices across multiple orders. On the other hand, it is useful to group orders by unit price, perhaps to answer the question, “How many did I sell at \$10 each versus \$12 each?” The unit price is, therefore, behaving as a dimension.

Dimension Table

Dimension's primary key

A dimension table is designed with one column serving as a unique primary key

primary key should not be the operational system's natural key

there will be multiple dimension rows for that natural key when changes are tracked over time.

natural keys for a dimension may be created by more than one source system, and these natural keys may be incompatible or poorly administered

rather than using explicit natural keys or natural keys with appended dates, you should create anonymous integer primary keys for every dimension.

These dimension surrogate keys are usually simple integers

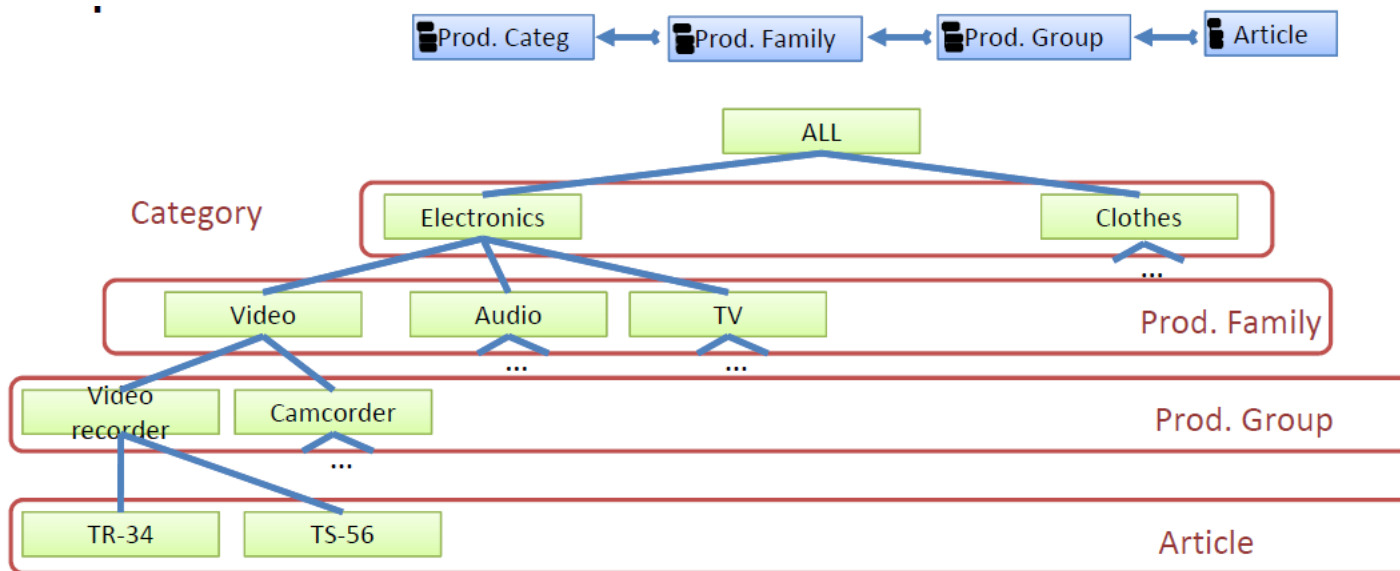
assigned in sequence, starting with the value 1, every time a new key is needed.

The date dimension is sometimes an exempt from the surrogate key rule

highly predictable and stable dimension can use a more meaningful primary key

Dimension Table

Hierarchies



Dimension Table

Levels

Hierarchy is utilised to support roll-up and drill-down operations

Path to lowest level element needs to be unique

Level should be associated with a unique key

natural or generated

often it's a composite key

Dimension Table

Levels

E.g. location:

city name -> province

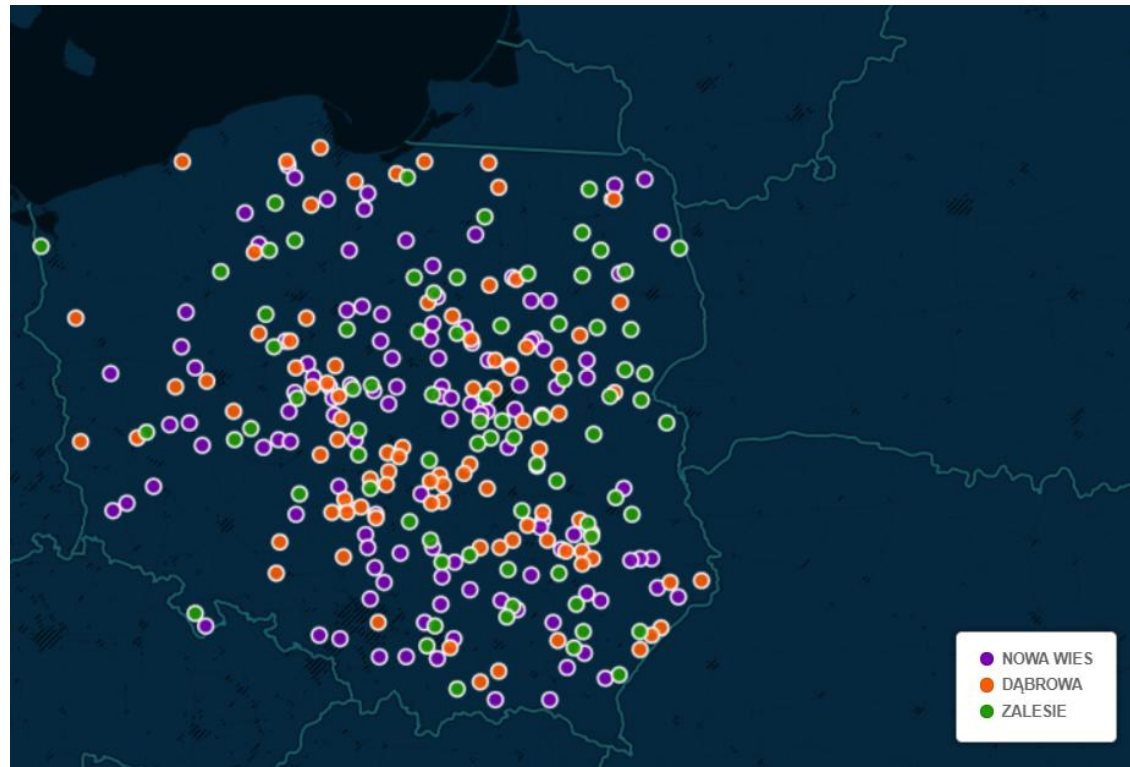
Nowa wieś

E.g. time:

month -> quarter -> year

January ?

Q1 ?



Dimension Table

Attribute hierarchies

are defined as a set of mandatory parent–child relationships among subsets of attributes.

offer a natural way to organize facts at successively deeper levels of detail

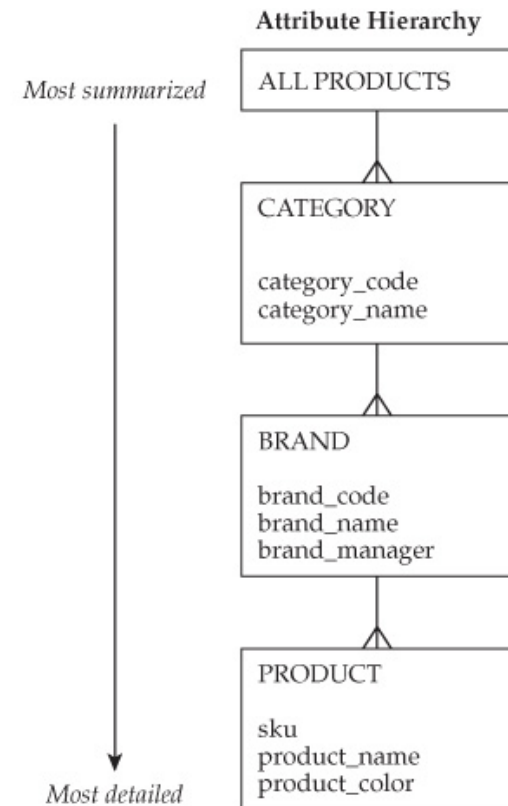
It is possible to describe a dimension table as a series of parent-child relationships among groups of attributes.

Days make up months, months fall into quarters, and quarters fall into years

When viewing a fact,

drilling down is accomplished by adding a dimension attribute from the next level down the hierarchy.

drilling up is achieved by removing attributes that belong to the current level of the hierarchy.





Note on hierarchies

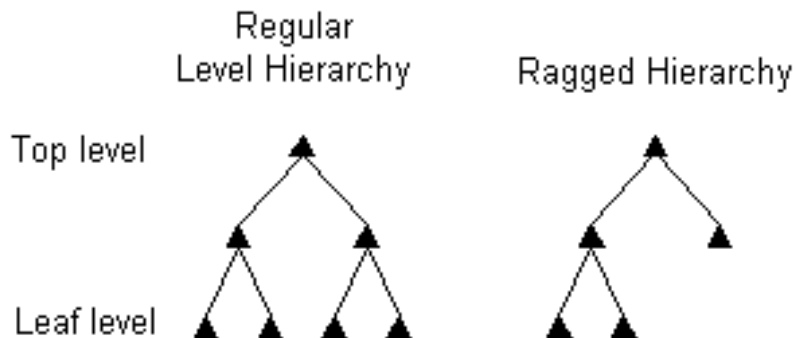
Dimension Table

Balanced hierarchies

A balanced hierarchy contains a fixed number of clearly defined levels

Easy to implement – horizontal view

E.g. time



Unbalanced hierarchies

An unbalanced hierarchy does not contain a fixed number of clearly defined levels

Recurrent, self-join

E.g. employee (parent-id as supervisor) – supervisor can have a supervisor

Needs to be supported by the tool

Roll-up and drill-down is not easy

Static set of attributes

E.g. supervisor-1, supervisor-2 ...

Hierarchy-bridge

Dimension Table

Instance hierarchies

are defined as relationships between rows of the dimension.

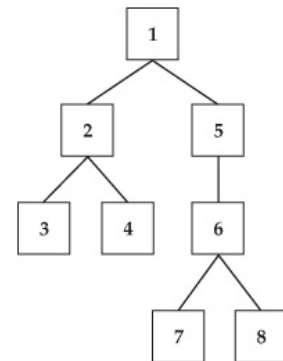
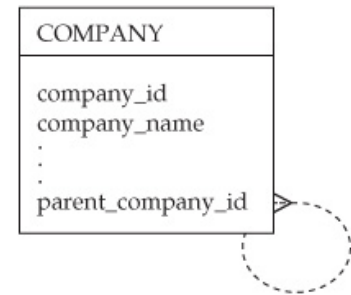
hierarchy exists among instances of dimensions

are recursive

Drilling process does not involve adding or removing attributes to our view of a fact

instances at each level share the same basic dimensional data

drilling requires tracing through the recursive relationships between instances



Unbalanced hierarchy

Self-join

Looking up or looking down occurs when users want to use this hierarchy to summarize facts.

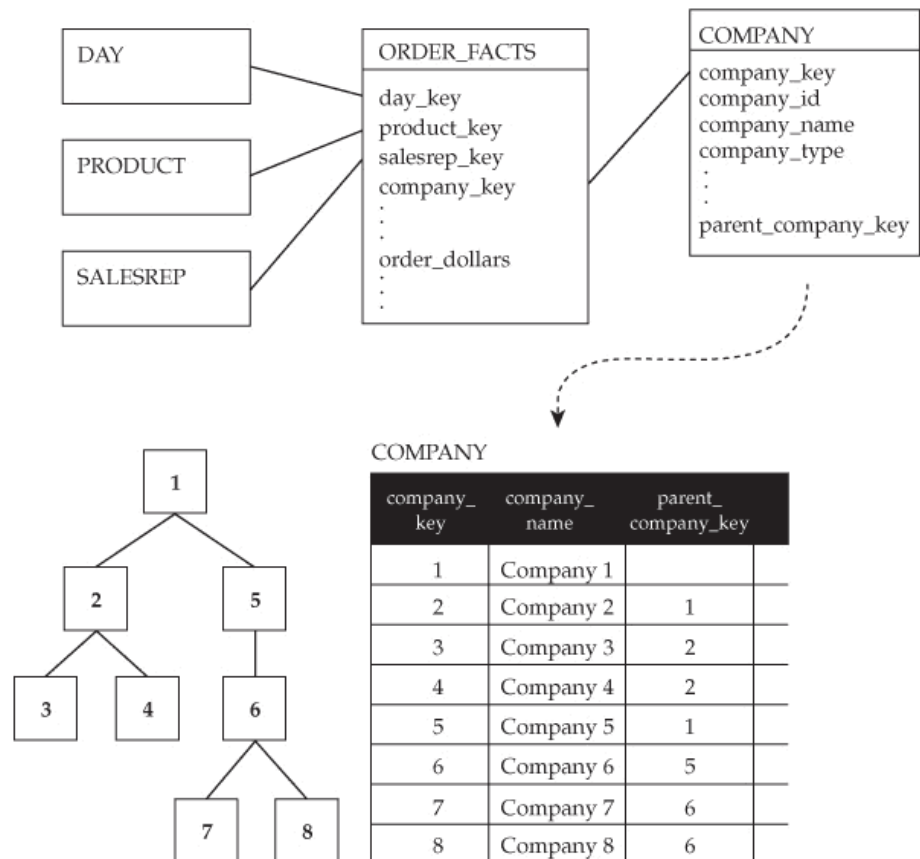
For example,

see all orders that take place below a particular company

looking down the hierarchy.

see all transactions that take place above a particular company,

looking up the hierarchy



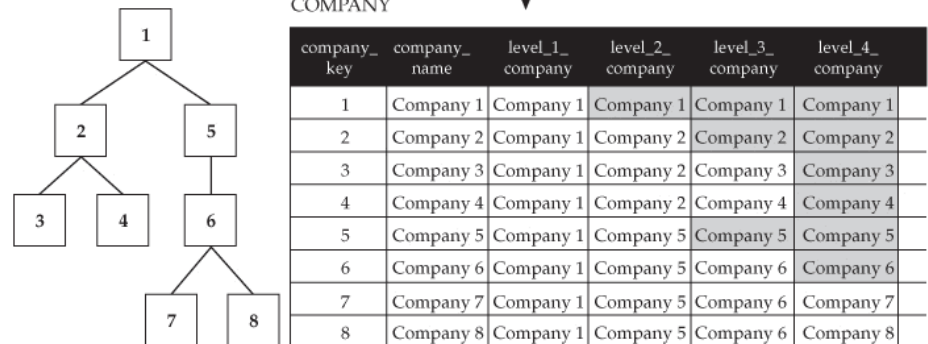
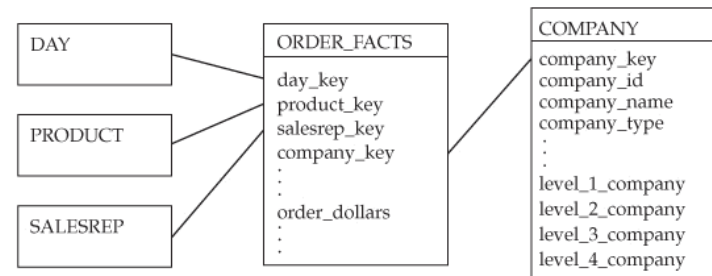
Unbalanced hierarchy

Flattening

The hierarchy is flattened by creating new attributes that represent a fixed number of levels.

Such a hierarchy looks and behaves like an attribute hierarchy.

It does not really solve the problems of looking up or looking down.



Unbalanced hierarchy

Bridge

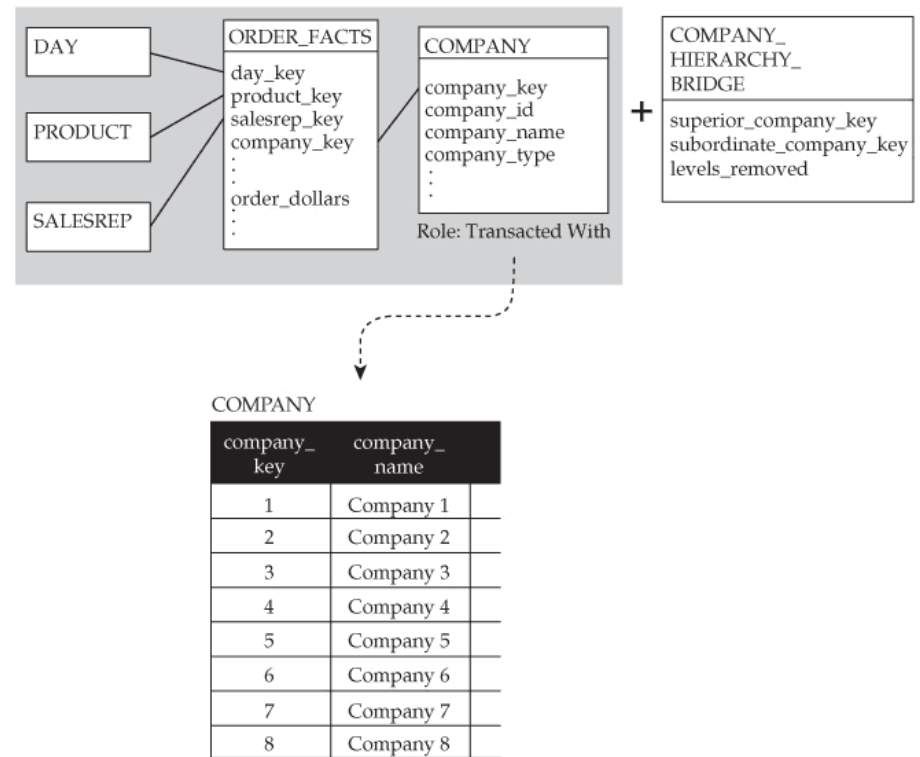
A hierarchy bridge table captures recursive instance relationships, so the dimension table will not need to be used

Each row of a hierarchy bridge table captures a relationship between a pair of rows in the dimension table.

Bridge table has two foreign key columns that refer to the dimension.

one represents the higher-level entity in the relationship,

and the other represents the lower-level entity.



COMPANY_ HIERARCHY_ BRIDGE
superior_company_key subordinate_company_key levels_removed

Unbalanced hierarchy

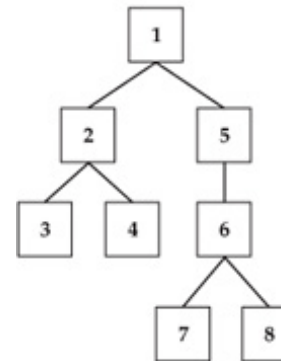
Bridge

There is more than one way to join a hierarchy bridge table to the other tables in a star schema.

The way in which you link the tables together in a query will depend on the kind of analysis being performed.

looking down the hierarchy

looking up the hierarchy



COMPANY_HIERARCHY_BRIDGE

superior_ company_ key	subordinate_ company_ key	levels_ removed
1	1	0
1	2	1
1	3	2
1	4	2
1	5	1
1	6	2
1	7	3
1	8	3
2	2	0
2	3	1
2	4	1
3	3	0
4	4	0
5	5	0
5	6	1
5	7	2
5	8	2
6	6	0
6	7	1
6	8	1
7	7	0
8	8	0

Dimension Table

Unbalanced/ragged hierarchies
– hierarchy bridge

Main tables:

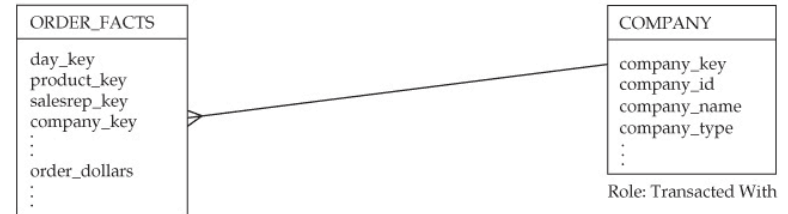
Fact – id – foreign key, value

Dimension – id – primary key, name

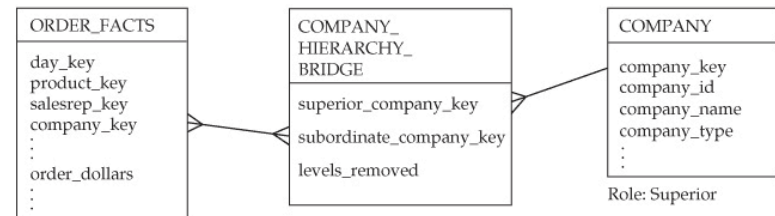
Additional tables:

Hierarchy table HT – parent_id, sub_id, levels removed

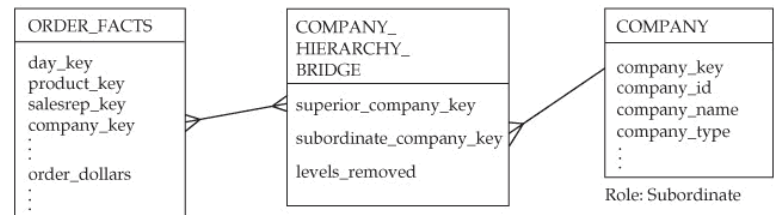
Transactions by Company



Transactions Beneath Companies (looking down)



Transactions Above Companies (looking up)

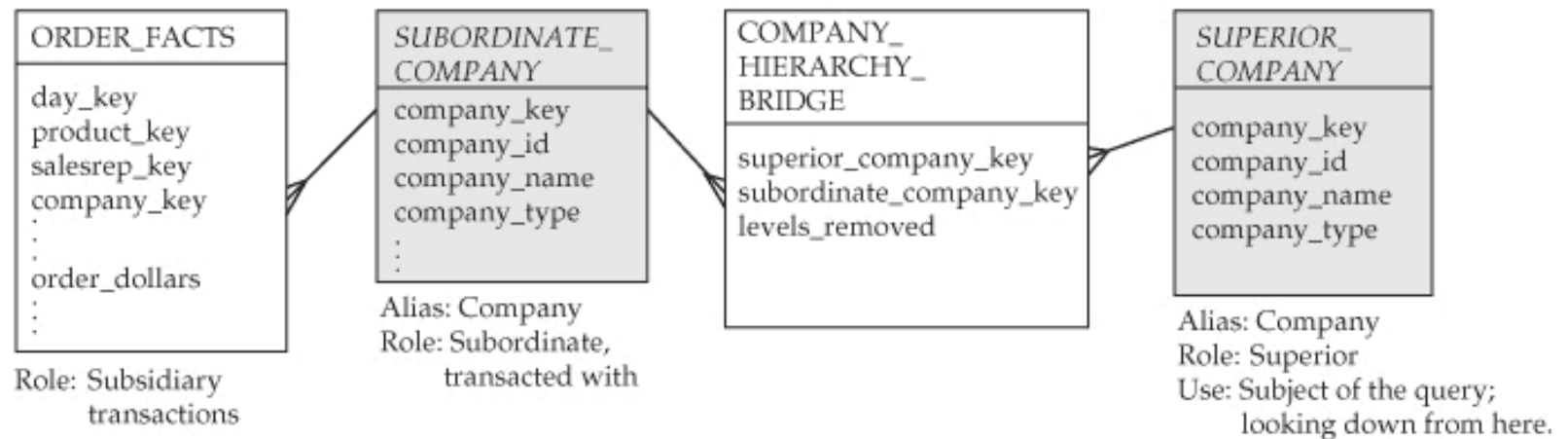


Unbalanced hierarchy

Bridge

The use of a hierarchy bridge will involve a many-to-many relationship.

If this is not supported by the used software – it requires decomposition into more join configurations.



Dimension Table

Unbalanced/ragged hierarchies
– hierarchy bridge

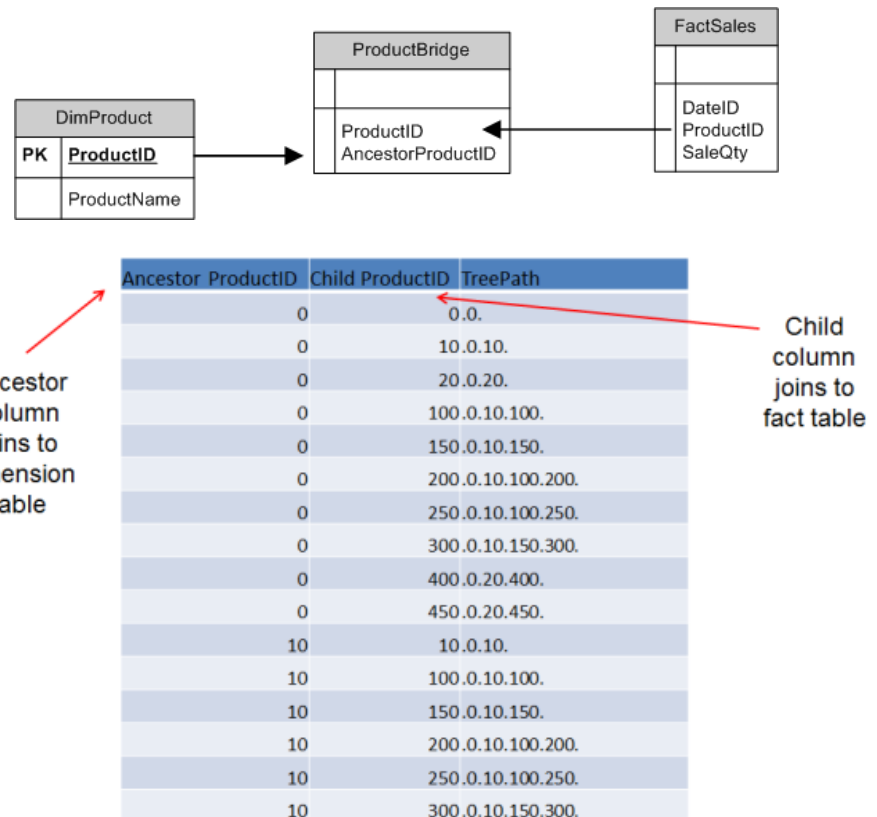
Main tables:

Fact – id – foreign key, value

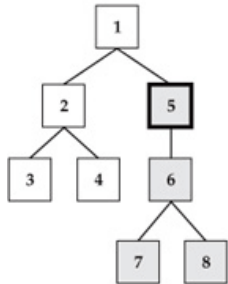
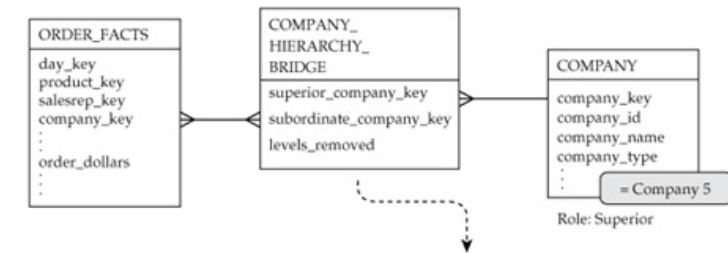
Dimension – id – primary key, name

Additional tables:

Hierarchy table HT – parent_id, sub_id, node (0/1), leaf (0,1), level

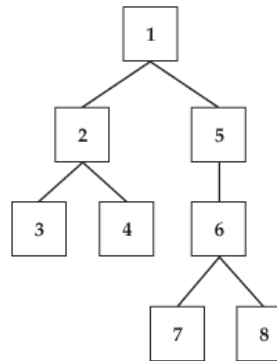
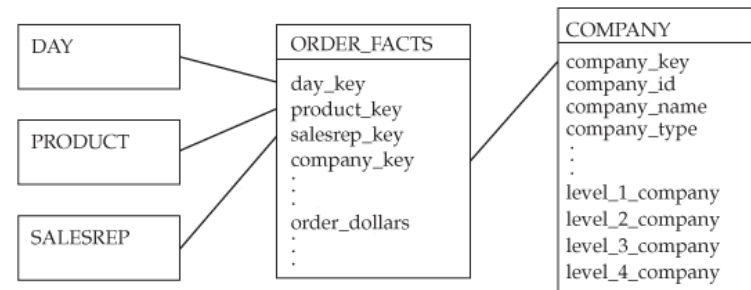


Dimension Table



COMPANY_HIERARCHY_BRIDGE

superior_company_key	subordinate_company_key	levels_removed
1	1	0
1	2	1
1	3	2
1	4	2
1	5	1
1	6	2
1	7	3
1	8	3
2	2	0
2	3	1
2	4	1
3	3	0
4	4	0
5	5	0
5	6	1
5	7	2
5	8	2
6	6	0
6	7	1
6	8	1
7	7	0
8	8	0



COMPANY

company_key	company_name	level_1_company	level_2_company	level_3_company	level_4_company
1	Company 1	Company 1	Company 1	Company 1	Company 1
2	Company 2	Company 1	Company 2	Company 2	Company 2
3	Company 3	Company 1	Company 2	Company 3	Company 3
4	Company 4	Company 1	Company 2	Company 4	Company 4
5	Company 5	Company 1	Company 5	Company 5	Company 5
6	Company 6	Company 1	Company 5	Company 6	Company 6
7	Company 7	Company 1	Company 5	Company 6	Company 7
8	Company 8	Company 1	Company 5	Company 6	Company 8



Types of dimensions

Dimension Table

Standard OLAP queries are fact-focused

- Query touches one fact table and its associated dimensions

Some types of analysis are dimension-focused

- Bring together data from different fact tables that have a dimension in common

- Common dimension used to coordinate facts

Dimension Table

Conformed dimension

Used by multiple fact tables

When dimensions do not conform

e.g., orders and shipments stars

implemented one at a time

if these individual stars do not share a common view of what a customer is, or what a product is?

while it is possible to study orders or shipments

it is not possible to compare them

Conformed dimensions are crucial in any data warehouse architecture that includes a dimensional component

key to long-term success

Dimension Table

Conformed dimension

- Used by multiple fact tables

- Need to maintain it's meaning in relation to different fact tables

- Need to be identical or one must be a part of the other

- Allow for a consistent data warehouse – among different facts

- Not easy to develop

Dimension Table

Conformed dimension

Dimensions are the key enablers of the drill-across activity that brings together information from different processes.

Traps

Same name - different meaning

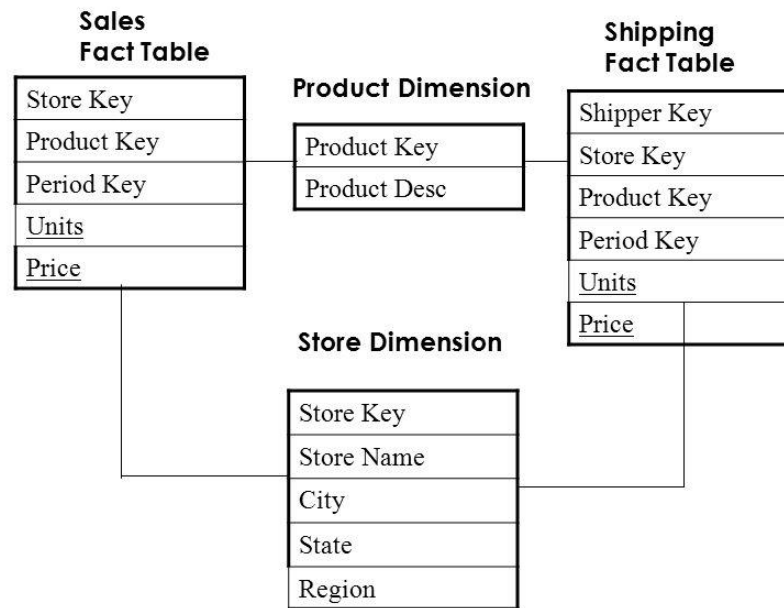
Time dimension - What about fiscal and calendar year?

Same data – different data organization

Different keys

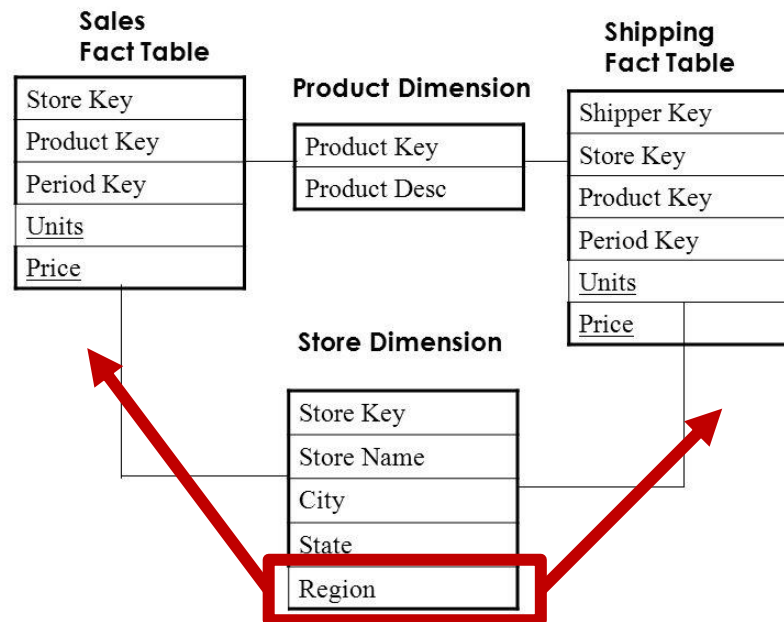
Dimension Table

More Examples on Fact-Constellation Schema

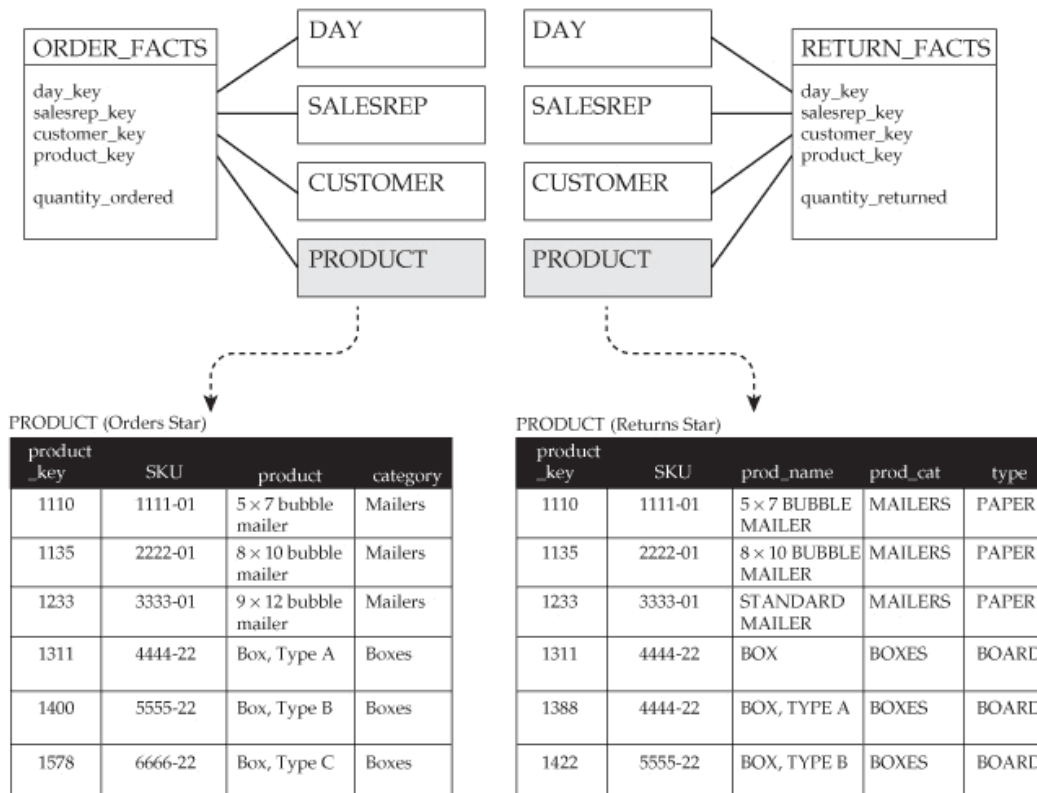


Dimension Table

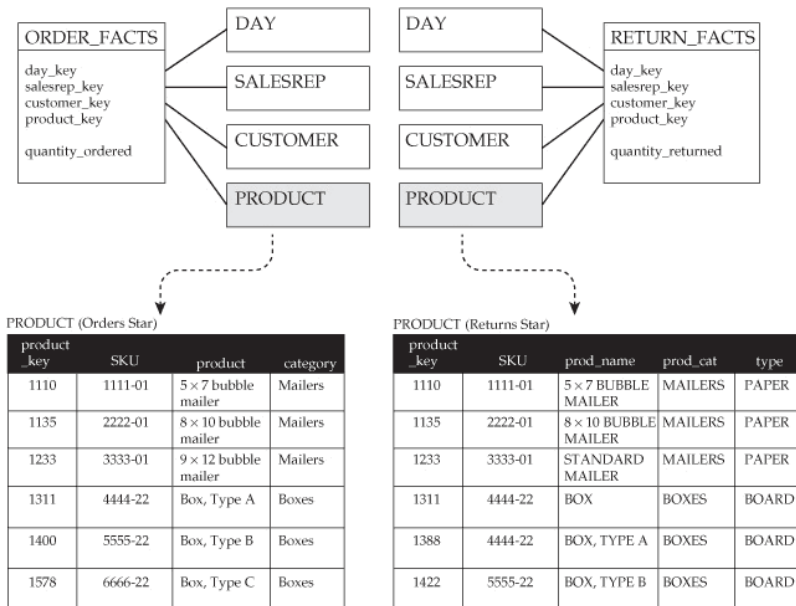
More Examples on Fact-Constellation Schema



Dimension Table



Dimension Table



differences

structure level

content level

workarounds are required to do cross-process analysis

risk of inconsistent and inaccurate results when applied incorrectly

additional knowledge is required

performance

Not every incompatibility can be overcome by a workaround

different definitions of a product

e.g., packaging differences

e.g., weekly / monthly data

not conformed

Dimension Table

Conformed dimension

To support successful drill-across comparisons, designers must avoid incompatibilities

two crucial parts to this sameness – structure and content

When dimension tables exhibit the compatibility necessary to support drilling across – they are conformed dimensions

Identical dimensions ensure conformance, but conformance can take several other forms as well

In a dimensional data warehouse

Conformed dimensions are a central feature of the design, providing enterprise capability

Dimension Table

Three types of “conformed” dimensions:

Dimension tables identical

shared dimension tables

Dimension table has subset of attributes from other dimension

Allows for roll-up dimensions at different grains

do not share a common surrogate key

“vertical sameness”

The smaller of the two dimensions is called a conformed rollup; the larger is called the base dimension.

Dimension table has subset of rows from the other dimension

When two tables share a set of common attributes, but one is not a perfect subset of the other, neither table can be described as a conformed rollup

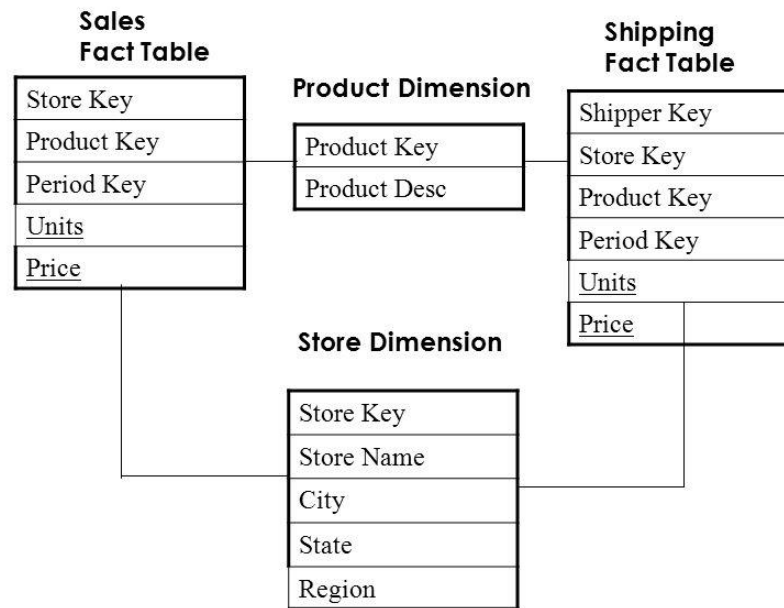
Can improve performance when many dimension rows are not relevant to a particular process

“horizontal sameness”

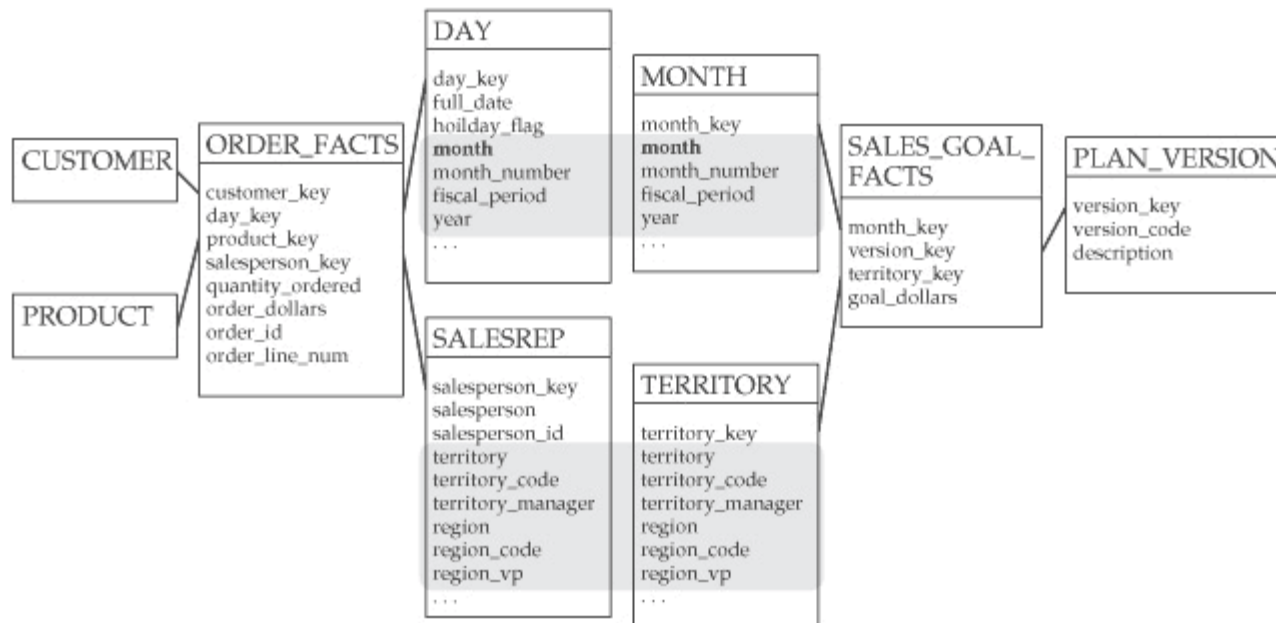
Overlapping Dimensions

Dimension Table

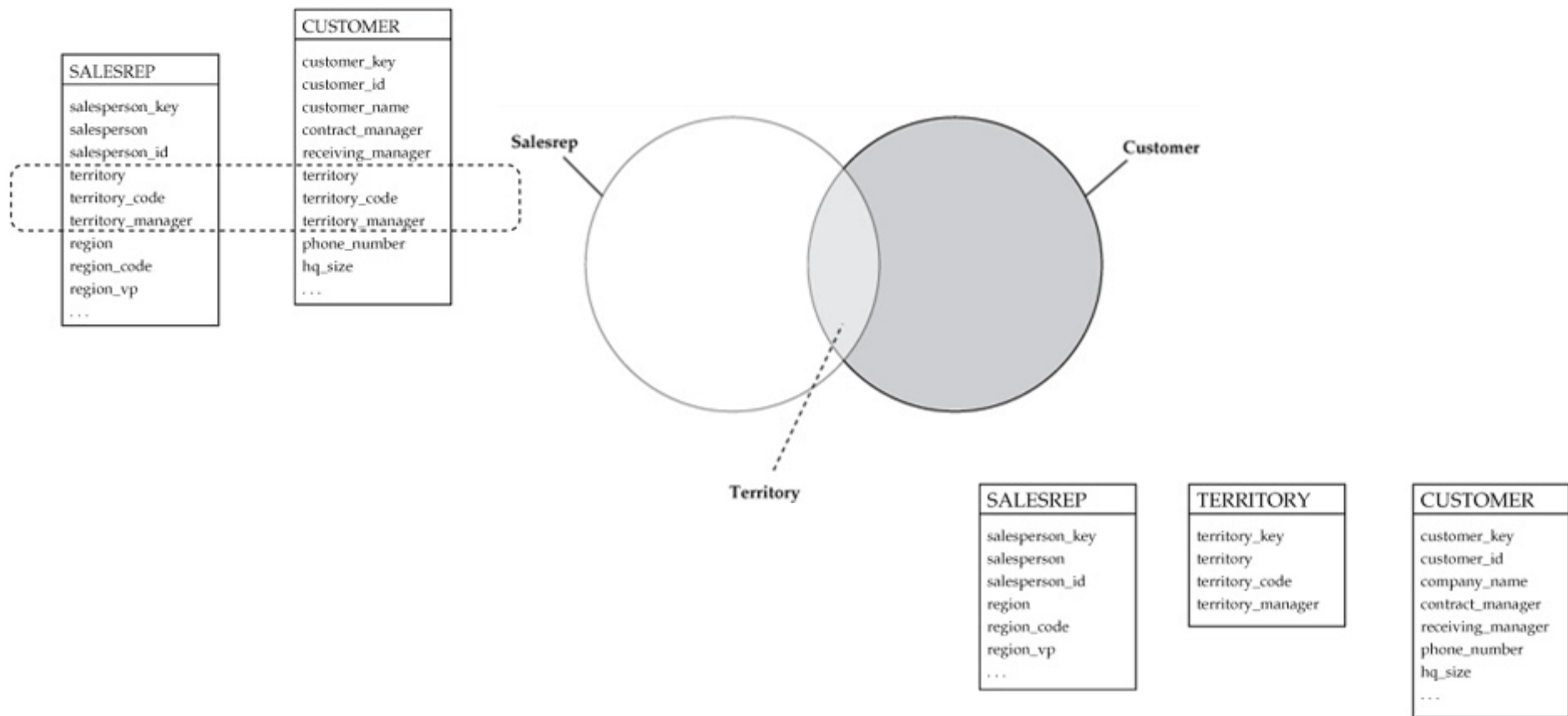
More Examples on Fact-Constellation Schema



Dimension Table



Dimension Table



isolated as a third dimension

Dimension Table

Conformed dimension

Dimensions are the key enablers of the drill-across activity that brings together information from different processes.

Drill-across

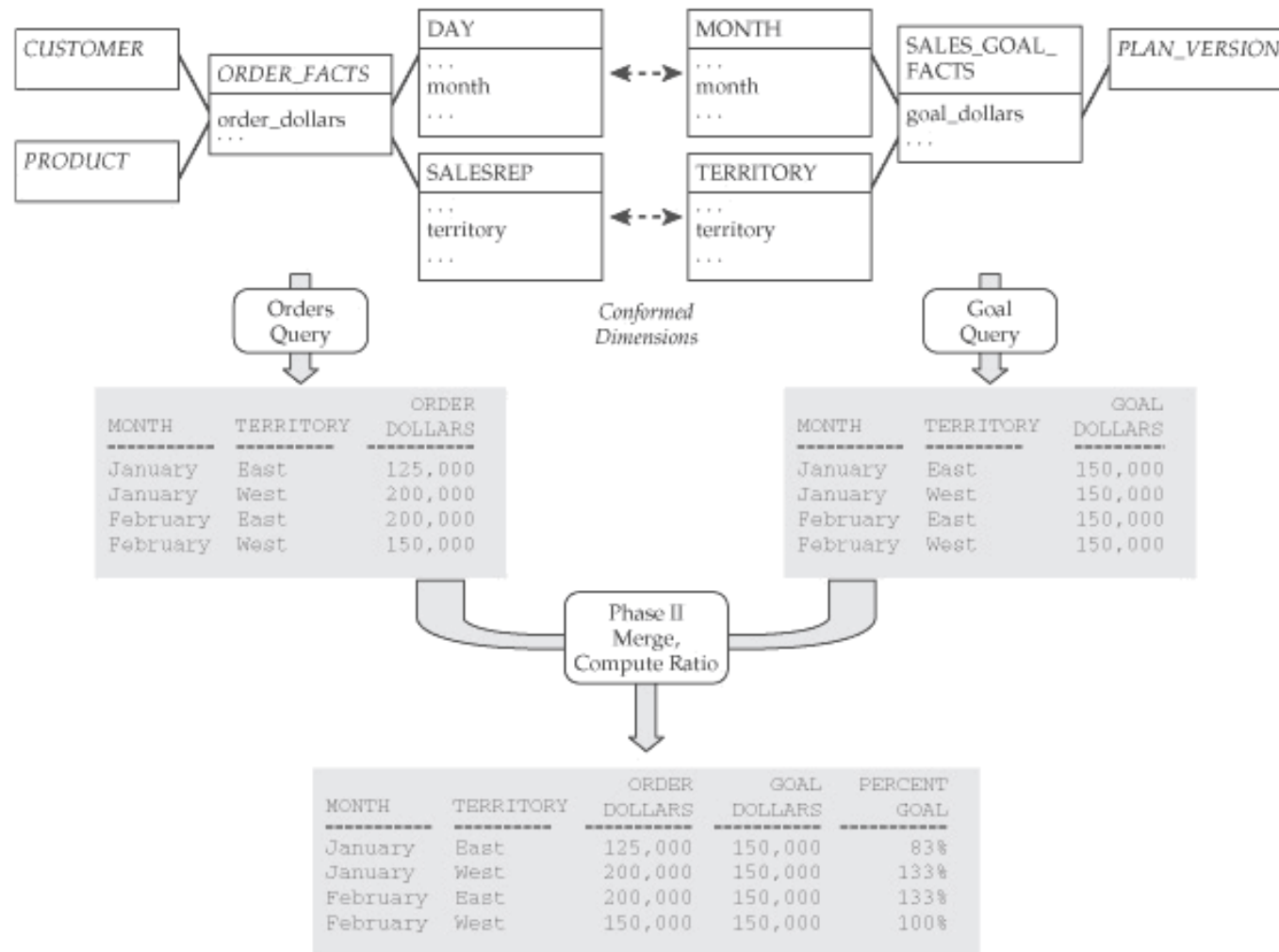
In the first phase

dimensions are used to define a common level of aggregation for the facts from each fact table queried

In the second phase

dimension's values are used to merge results of these queries

Dimension Table



Dimension Table

Conformed dimension

POS Sales fact recorded at fine-grained detail

Date, Product, Store, Promotion

Monthly sales forecasts

Predicted sales for each brand in each district in each month

Month, Brand, District

Question - How did actual sales diverge from forecasted sales in given month (month, year)?

Drill-across between Forecast and Sales

Dimension Table

Drill-across example

Sales fact with dimensions (Date, Customer, Product, Store)

CustomerSupport fact with dimensions (Date, Customer, Product, ServiceRep)

Question: How does frequency of support calls by California customers affect their purchases of Product X?

Step 1: Query CustomerSupport fact

Group by Customer SSN ; Filter on State = California ; Compute COUNT ; Query result has schema (Customer SSN, SupportCallCount)

Step 2: Query Sales fact

Filter on State = California, Product Name = Product X ; Compute SUM(TotalSalesAmt) ; Query result has schema (Customer SSN, TotalSalesAmt)

Step 3: Combine query results

Join Result 1 and Result 2 based on Customer SSN ; Group by SupportCallCount ; Compute COUNT, AVG(TotalSalesAmt)

Dimension Table

Conformed dimension and ETL

When a conformed dimension is implemented as separate physical tables, a single ETL process should be responsible for updating it

May be achieved by updating a master table first, then replicating it to the separate physical locations.

This practice guarantees that the replicas will be identical, cuts down on duplicative processing, and guarantees accurate results when the replicas are used for analysis.

For larger tables, replication may not be practical.

a single ETL process should identify new and changed rows, perform key management once, and apply the changes to each replica.

Dimension Table

Conformed dimension and ETL

To guarantee that the instance values of the conformed rollup match those of the base dimension is to designate the base dimension as its source.

This ensures consistent computation of value instances based on the source data.

Developers may choose to process the base dimension first, then review the new and changed rows to process the rollup.

Alternatively, they may choose to build a single routine that processes source data and applies new and changed rows to the base and rollup simultaneously.

Dimension Table

Document dimensional conformance across fact tables or subject areas using matrix diagrams – conformance matrix

columns represent the core conforming dimensions

rows represent various processes or fact tables

Conformance matrix serves as a blueprint for implementation

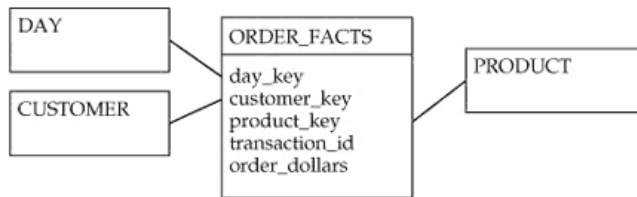
Degenerate dimensions
should be included

if they might serve as the basis
for drilling across

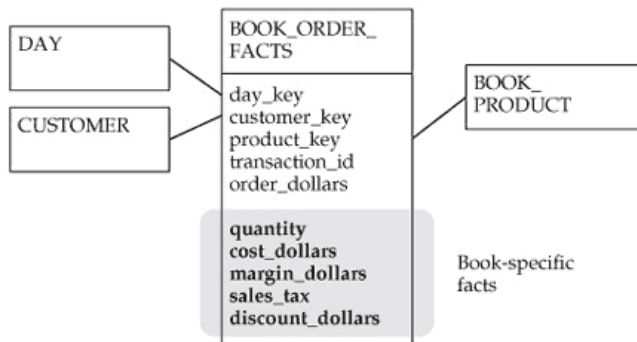
Not every dimension needs to
be included

	day			product		salesrep			warehouse	order_line
	day	month	quarter	product	category	salesrep	territory	region		
order_facts	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
shipment_facts	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
return_facts	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
inventory_facts	✓	✓	✓	✓	✓				✓	
receivables_facts	✓	✓	✓						✓	✓
sales_goal_facts		✓	✓				✓	✓		
demand_forecast_facts			✓		✓			✓		

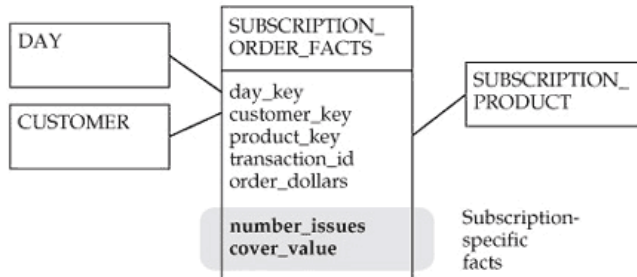
Core and Custom



Core Star



Custom Stars



A core star includes all common attributes and supports analysis across all types.

Type-specific custom stars include all the core attributes, plus any type-specific attributes

Dimension Table

Overlapping Dimensions

non-identical dimension tables may conform through a set of overlapping attributes

Overlapping dimension tables are usually maintained by separate ETL processes.

there is a risk, however small, that they will be loaded with inconsistent values.

For these reasons, designers generally try to avoid overlapping dimensions.

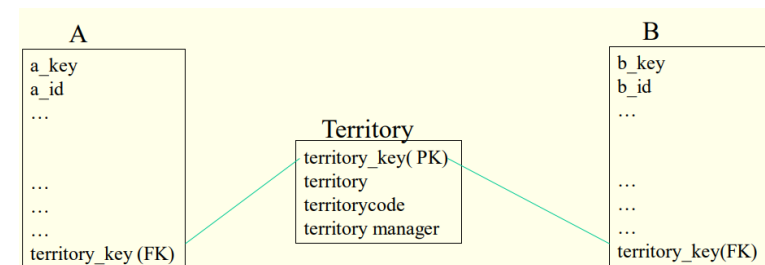
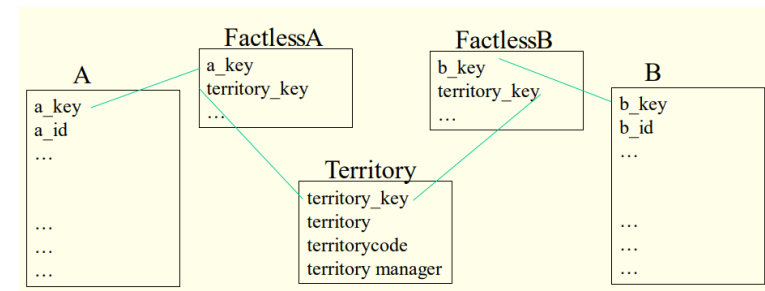
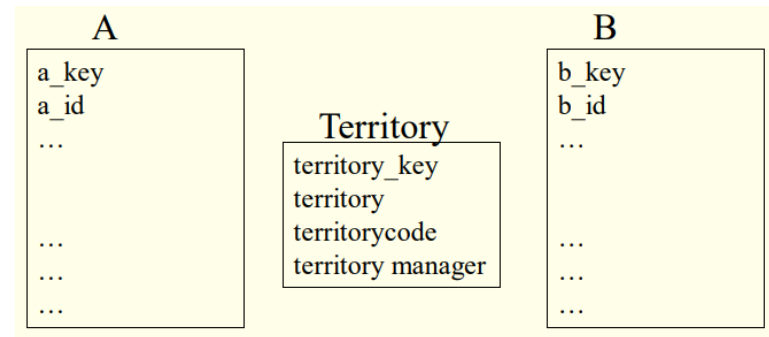
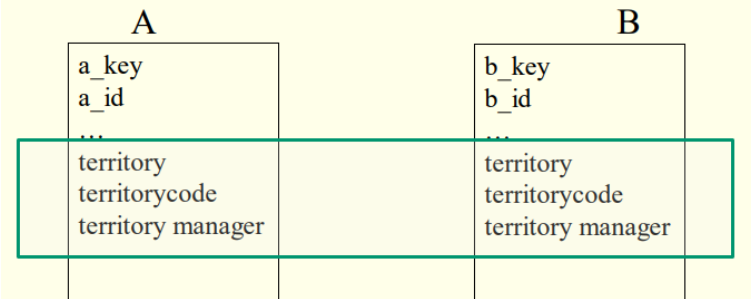
isolated third dimension table

third dimension table with relationship tracking

factless fact tables tracking the relation

outrigger – snowflake third dimension table

Overlapping Dimensions



Dimension Table

Abstract dimension – Junk dimension:

Transactional business processes typically produce several miscellaneous flags and indicators

low-cardinality

Kimball suggest using less than 26 dimensions

How to handle this situation ?

Age group

Customer's status

Dimension Table

Abstract dimension – Junk dimension:

Rather than making separate dimensions for each flag and attribute, you can create a single junk dimension

combining them together

Separate dimension table with all this information

Surrogate keys are required

This dimension does not need to be the Cartesian product of all the attributes' possible values

should only contain the combination of values that actually occur in the source data

Dimension Table

Abstract dimension – Junk dimension:

Order Indicator Key	Payment Type Description	Payment Type Group	Order Type	Commission Credit Indicator
1	Cash	Cash	Inbound	Commissionable
2	Cash	Cash	Inbound	Non-Commissionable
3	Cash	Cash	Outbound	Commissionable
4	Cash	Cash	Outbound	Non-Commissionable
5	Visa	Credit	Inbound	Commissionable
6	Visa	Credit	Inbound	Non-Commissionable
7	Visa	Credit	Outbound	Commissionable
8	Visa	Credit	Outbound	Non-Commissionable
9	MasterCard	Credit	Inbound	Commissionable
10	MasterCard	Credit	Inbound	Non-Commissionable
11	MasterCard	Credit	Outbound	Non-Commissionable
12	MasterCard	Credit	Outbound	Commissionable



Model ID	Processor	Tower	Purpose
1001	5950X	Compact	Workstation
1002	5950X	Full	Workstation
1003	5950X	Compact	Gaming
1004	5950X	Full	Gaming
1005	i9	Compact	Workstation
1006	i9	Full	Workstation
1007	i9	Compact	Gaming
1008	i9	Full	Gaming

Figure 6-8: Sample rows of order indicator junk dimension.



Dim_ProductType	
Id	ProductType
1	For sale
2	For sample
3	Dummy piece
4	Show piece
5	For use
6	Not applicable

Dim_TaxInformation	
Id	TaxInformation
1	Applicable
2	Not applicable
3	12% Tax on MRP

Dim_ProductQuality	
Id	QualityStatus
1	High
2	Low
3	Medium

Dim_Expiry	
Id	IsExpiredText
1	Yes
2	No
3	Not applicable

Dim_StorageType	
Id	Storage
1	Hot
2	Cold
3	Not applicable

Dim_DeliveryMode	
Id	DeliveryMode
1	Dealer
2	Agent
3	Direct

Dimension Table

Abstract dimension – Junk dimension:

- Best suited are attributes with a predefined set of static values

- Too many attributes might require introducing multiple abstract dimensions

- Can be prepared outside of the ETL process

Traps:

- Treating abstract attributes as a part of a larger dimension*

 - E.g., Age group, patient's type

- Treating abstract dimension as a part of a fact table*

 - Inefficient

Dimension Table

Degenerate dimension

For example, when an invoice has multiple line items

the line item fact rows inherit all the descriptive dimension foreign keys of the invoice

the invoice is left with no unique content

but the invoice number remains a valid dimension key

for fact tables at the line item level

How this is different from abstract dimension ?

Dimension Table

Degenerate dimension

Dimension that is stored within the fact table

High-cardinality

Often nearly 1-1 with facts

Sometimes a dimension is defined that has no content except for its primary key

Degenerate dimension is placed in the fact table with the explicit acknowledgment that there is no associated dimension table

Degenerate dimensions are most common with transaction and accumulating snapshot fact tables

Dimension Table

Degenerate dimension

Consider an example – stomatologist

Fact:

a single visit/service

associated with a particular treatment

Dimensions:

Patient

Time

Service details

What about situations that a certain treatment requires multiple visits/services?

Dimension Table

Degenerate dimension

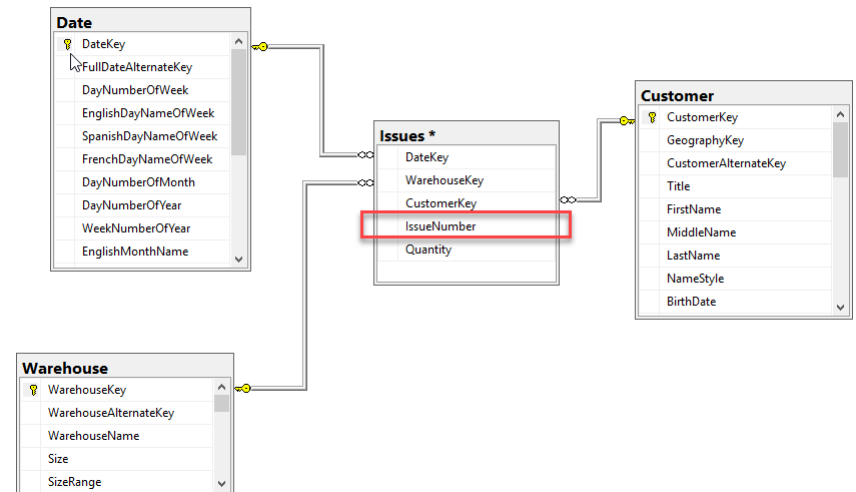
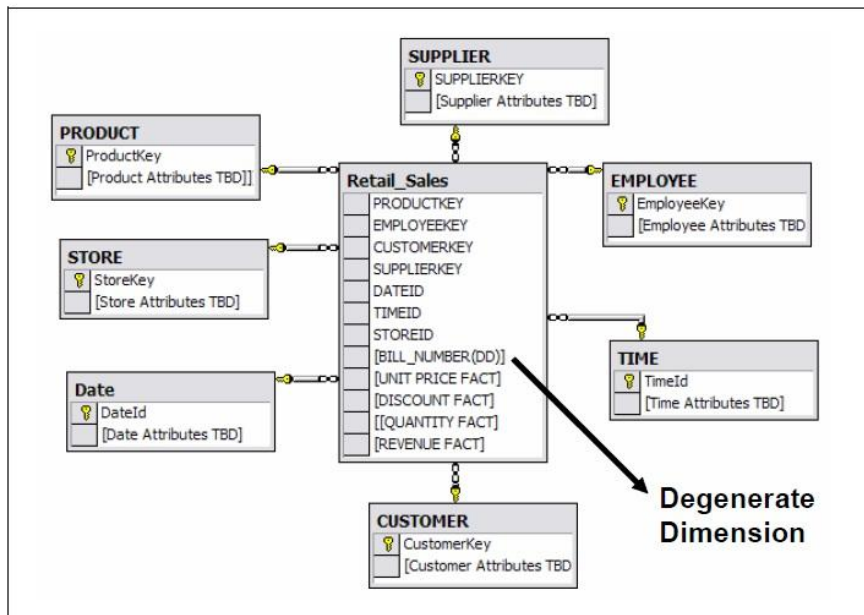


Figure 6-26 Degenerate dimension

Source: Dimensional Modeling: In a Business Intelligence Environment, Page 241

Dimension Table

Role-playing dimension:

A single physical dimension can be referenced multiple times in a fact table

with each reference linking to a logically distinct role for the dimension.

each foreign key refers to a separate view of the date dimension so that the references are independent

separate dimension views are called roles

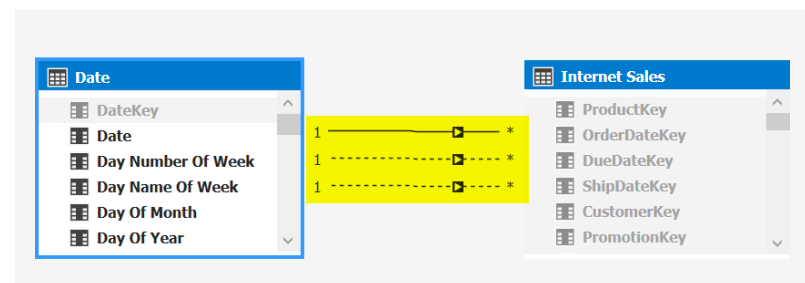
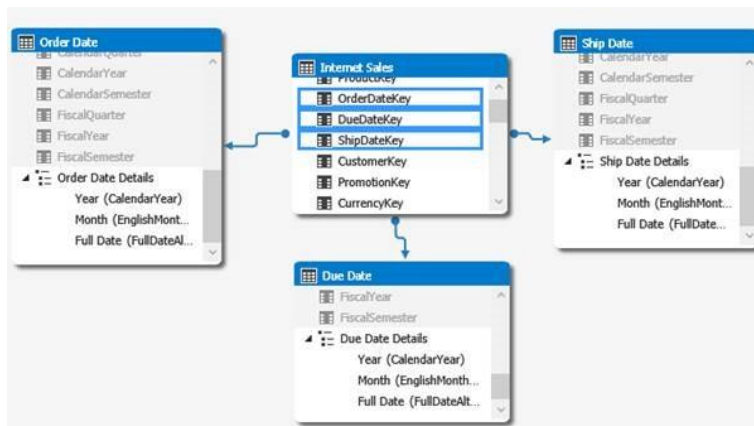
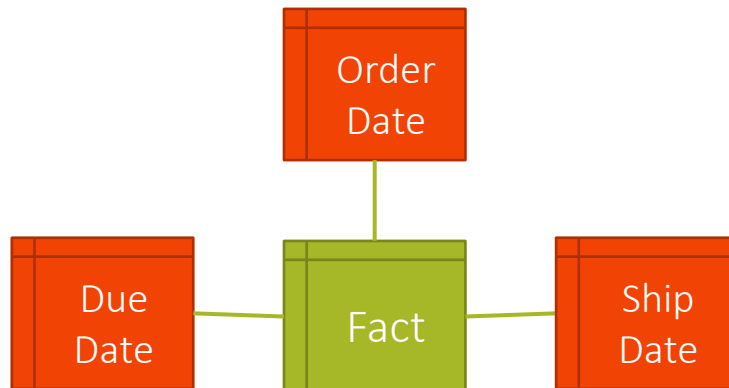
role-playing dimension

For instance, a fact table can have several dates

each of which is represented by a foreign key to the date dimension.

Dimension Table

Role-playing dimension:



Behavioral Dimensions

A behavioral question is one that groups or filters facts based on the past behavior of members of a dimension.

Behavioral dimensions transform facts into dimensions, enabling analysis without complex queries or intensive processing

Example

Are customers who generate over \$1 million in orders receiving better discounts than those who generate \$500,000 or less?"

We need to determine who those customers are first

Answering a question that groups or filters facts based on past behaviour goes beyond the capabilities of the basic SELECT/GROUP BY query

To answer these questions, two major steps are required:

1. Identify past behavior for each member of the dimension in question.
2. Use this information as part of a query studying current behavior.

Suggested Technique:

add columns to a dimension to track past behaviour.

Dimension Table

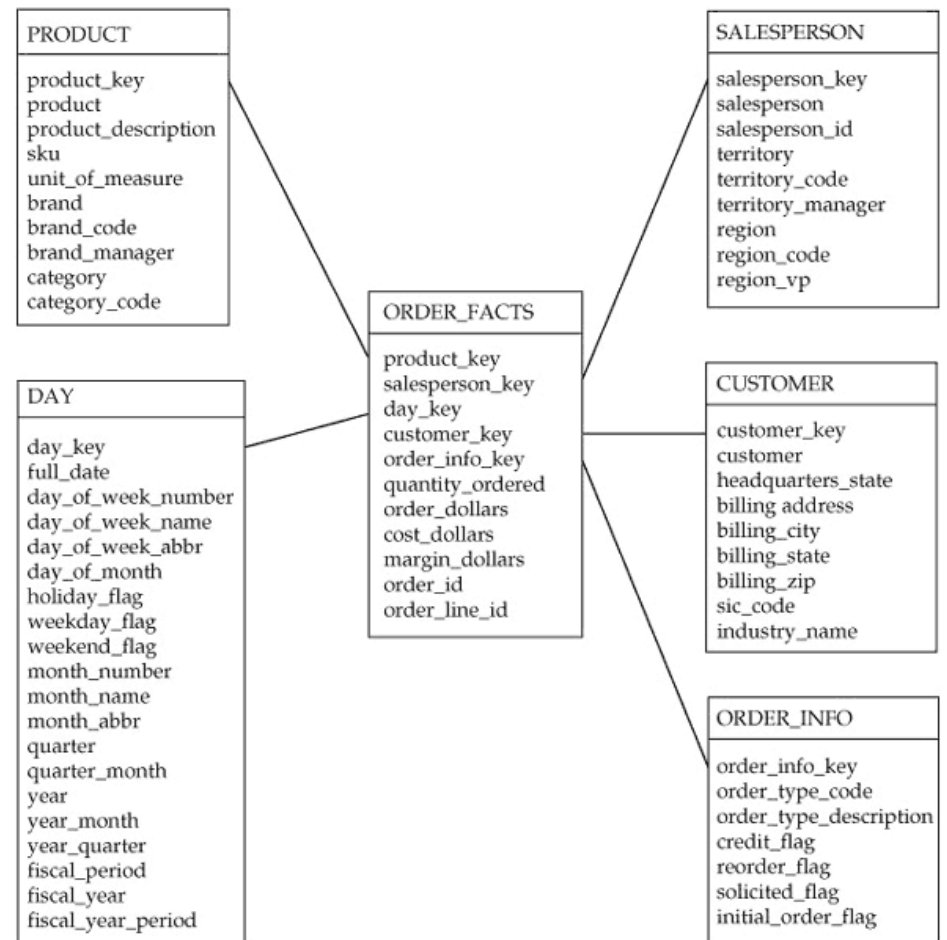
Identify dimensions

Abstract

Degenerate

Role-playing

Conformed





Note on dimensions

Dimension Table

Dimension tables with a large number of attributes maximize analytic value.

They can be thought of as wide.

In addition to storing common attributes,

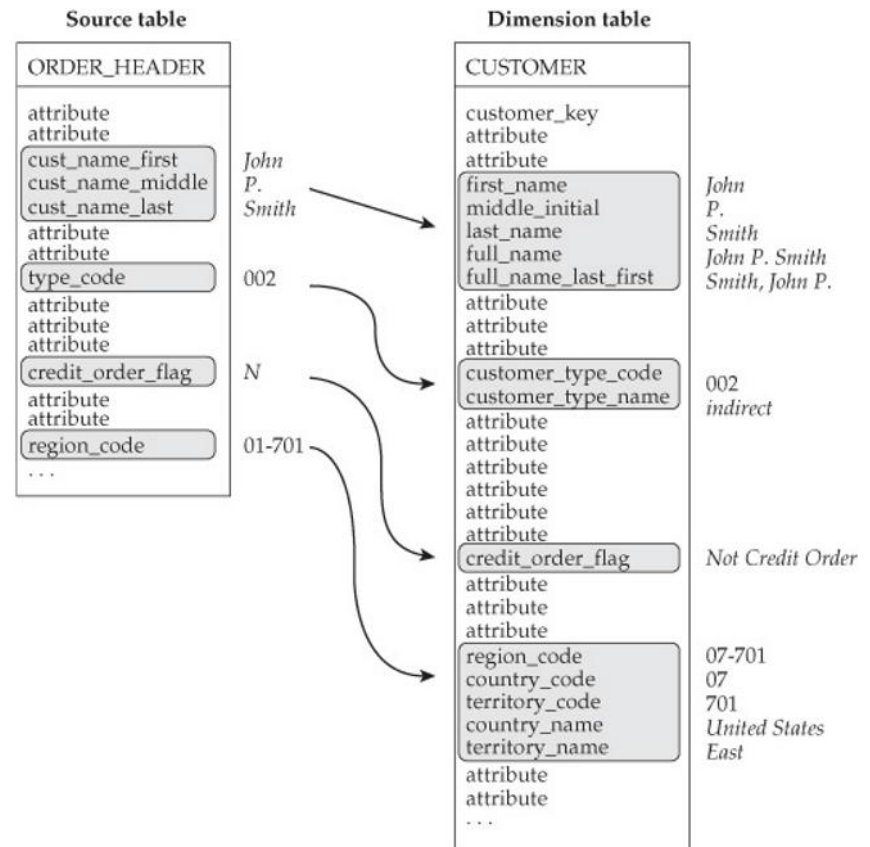
dimension tables store commonly used combinations of attributes.

Codes may be supplemented with corresponding description values.

Flags are translated from Boolean values into descriptive text,

multi-part fields are both preserved and broken down into constituent pieces.

It is also important not to overlook numeric attributes that can serve as dimensions.



Dimension Table

Why use snowflake?

Attribute groups

E.g. registered / unregistered users

Multiple hierarchies

E.g. customer – city – country

E.g. customer – gender - married

One table for one level

Dimension Table

In general, dimensional designers must resist the normalization urges

- instead denormalize the many-to-one fixed depth hierarchies into separate attributes on a flattened dimension row

Dimension denormalization supports dimensional modeling's objectives

- simplicity and speed

Dimension Table

Goal for dimensional modeling:

Surround facts with as much context (dimensions) as possible

Hint:

redundancy may be ok (in well-chosen places)

But you should not try to model all relationships in the data

(unlike E/R and OO modeling!)

Dimension Table

Goal of the designer is to make all values

Verbose (use full words)

Cryptic abbreviations, true/false flags, and operational indicators should be supplemented with full text words that have meaning when independently viewed

Descriptive

Complete – no missing values

*It is recommended to substitute with a descriptive string
such as Unknown or Not Applicable*

Quality assured

Indexed, for those most heavily used

Documented in metadata



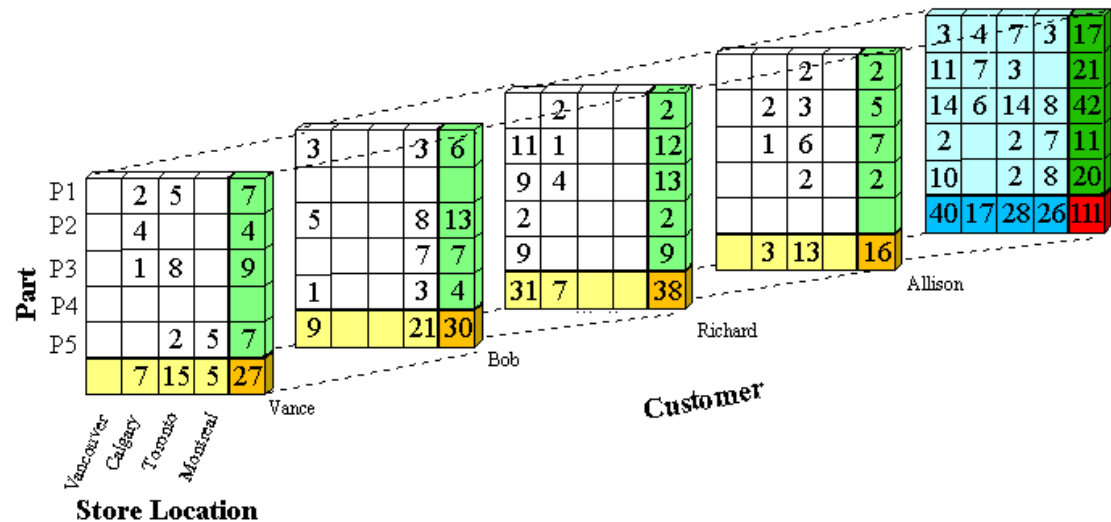
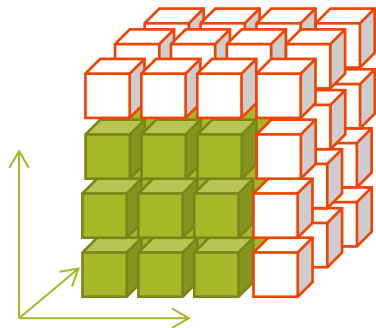
Note on measures

Aggregates

Multi-dimensional data

Aggregates

together with measure values we store summarizing information



Measures

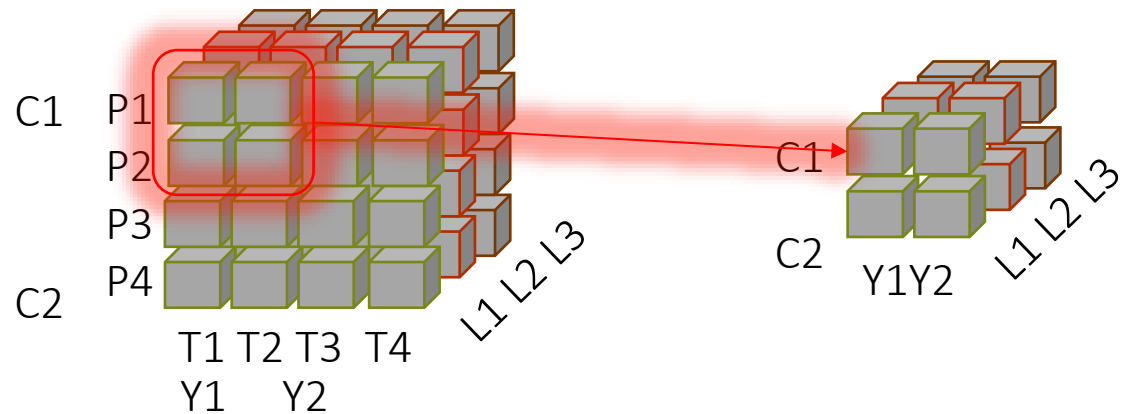
Measure has two components

- Numerical value: (sales price)

- Aggregation formula (SUM): used for aggregating/combining a number of measure values into one

Measure value is determined by dimension value combination

Measure value should be meaningful for all aggregation levels



Measures

Measure types

Additive

measures that can be added across any dimension.

For example total sales amount can be added across all dimensions

Non-additive

measures that cannot be added across any dimension.

For example product's margin

Semi-additive

measures that can be added across some dimensions.

For example the bank account balance is simply a snapshot in time and cannot be added over time. However you could add multiple accounts of the same customer to get the total balance for that customer.

Measures

Additive facts are easy to work with

Summing the fact value gives meaningful results

Additive facts: Quantity sold, Total sales

Non-additive facts:

Averages (average sales price, unit price)

Percentages (% discount)

Ratios (gross margin)

Count of distinct products sold

Measures

Measures can be organized into three categories based on the kind of aggregate functions used:

Distributive:

An aggregate function is distributive if it can be computed in a distributed manner. Distributive measures can be computed efficiently because of the way the computation can be partitioned.

Algebraic:

Like distributive aggregate functions, algebraic functions can be evaluated in different stages, but requires a consolidating function at the final stage to accumulate the result generated in the previous stages.

Holistic:

An aggregate function is holistic if there is no constant bound on the storage size needed to describe a subaggregate.

There does not exist an algebraic function with M arguments (where M is a constant) that characterizes the computation.

Measures

Distributive measure

The result derived by applying the function to n aggregate values is the same as that derived by applying the function on all the data without partitioning

A function $F()$ is distributive if there is a scalar function $G()$ such that

$$F(\{X_{i,j}\}) = G(\{F(\{X_{i,j}|i = 1, \dots, I\})|j = 1, \dots, J\})$$

Algebraic measure

Can be computed by an algebraic function with M arguments (where M is a bounded integer), each of which is obtained by applying a distributive aggregate function

A function $F()$ is algebraic if there is a m -valued function $G() \neq F()$ and m -valued function $H() \neq F()$ such that

$$F(\{X_{i,j}\}) = H(\{G(\{X_{i,j}|i = 1, \dots, I\})|j = 1, \dots, J\})$$

$G()$ computes aggregates for each group

$H()$ consolidates results for each group

Holistic

There is no constant bound on the storage size needed to describe a subaggregate

Measures

Distributive measure

A measure is distributive if it is obtained by applying a distributive aggregate function.

For example, `count()` can be computed for a data cube by first partitioning the cube into a set of subcubes, computing `count()` for each subcube, and then summing up the counts obtained for each subcube.

Sum, min, max

Algebraic measure

A measure is algebraic if it is obtained by applying an algebraic aggregate function.

For example, `avg()` can be computed by `sum()/count()`, where both `sum()` and `count()` are distributive aggregate functions

min_N, max_N, standard deviation

Holistic

A measure is holistic if it is obtained by applying a holistic aggregate function.

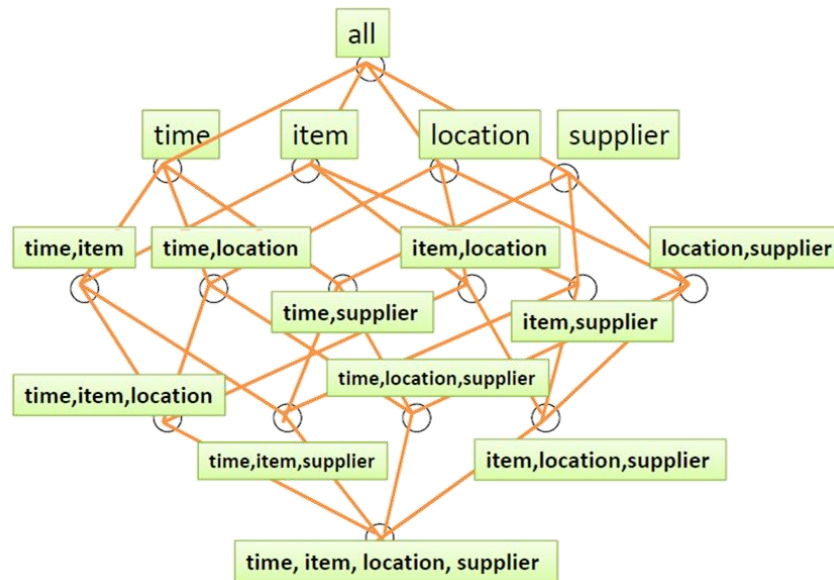
For example, mode (“most popular item”) can be computed only by `sum()/count()`, where both `sum()` and `count()` are distributive aggregate functions

Mode, median, rank

Multi-dimensional data

Data cube

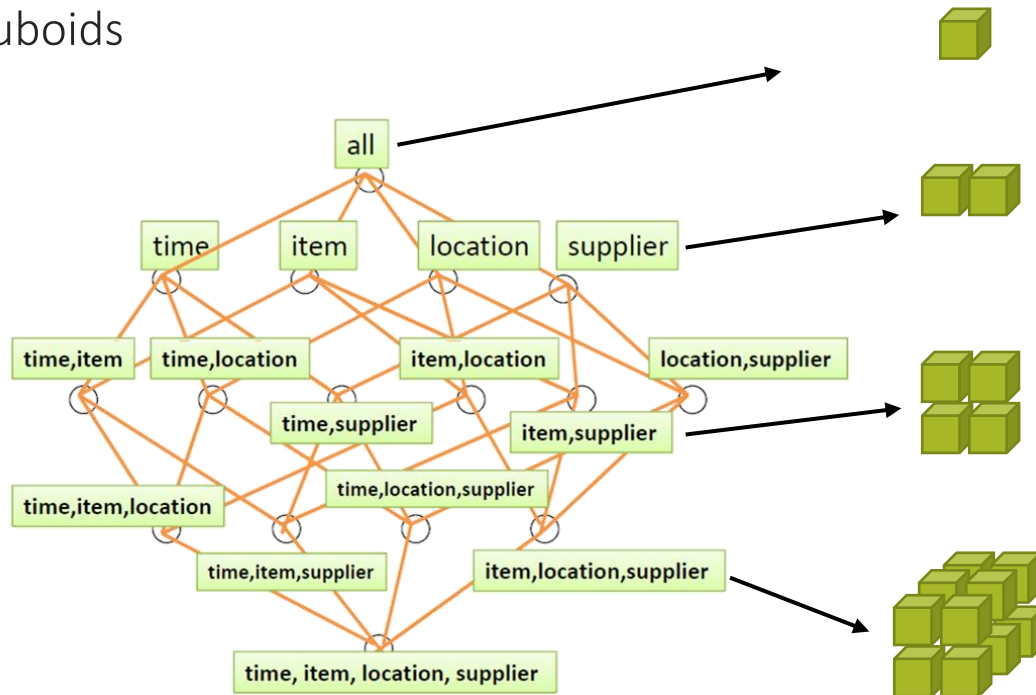
Lattice of cuboids



Multi-dimensional data

Data cube

Lattice of cuboids



Multi-dimensional data

Lowest level of summarization is called the base cuboid

In general N-D, where N is the number of dimensions

E.g. base cuboid for the given *time*, *item*, *location*, and *supplier* dimensions - 4-D cuboid

M-D cuboid

M dimensions, with N-M summaries

E.g. 3-D cuboid for time, item, and location - summarized for all suppliers

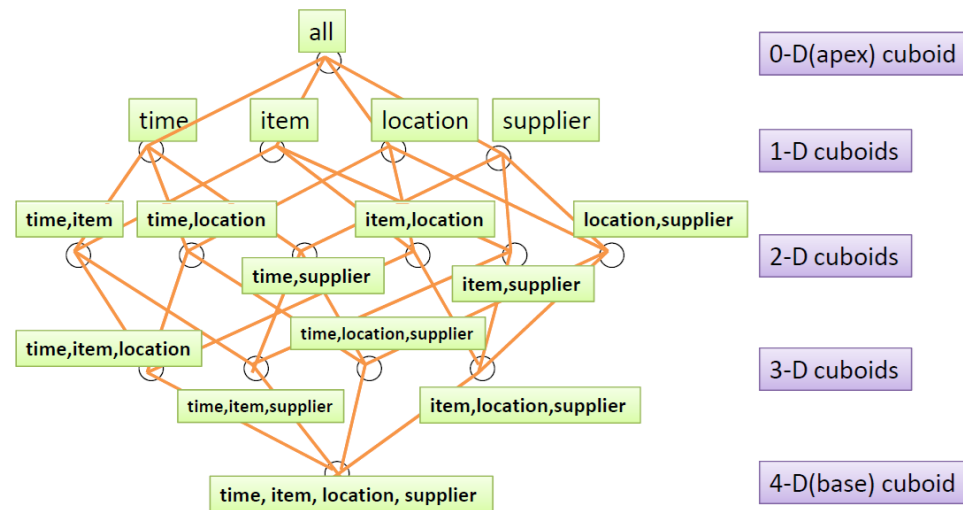
The 0-D cuboid

holds the highest level of summarization

called the apex cuboid

E.g. total sales summarized over all four dimensions

typically denoted by all



Multi-dimensional data

2 dimensions

location

country

state

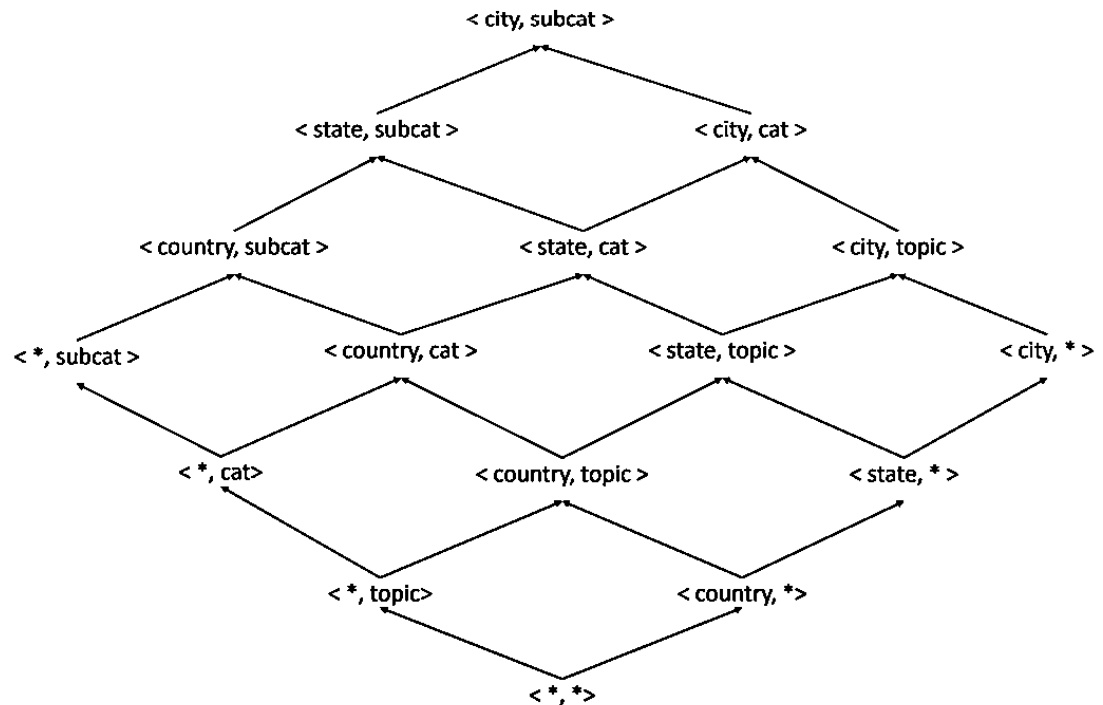
city

topic

topic

cat

subcat



Adamson C.

Star Schema The Complete Reference
McGraw-Hill, 2010

Jensen C.S., Pedersen T.B., Thomsen C.,
Multidimensional Databases and Data Warehousing,
Morgan & Claypool Publishers series "Synthesis
lectures on data management", 2010

Inmon W.,
Building the Data Warehouse,
John Wiley & Sons, New York 2002

Claudia Imhoff, Nicholas Galletto, Jonathan G.
Geiger,
Mastering Data Warehouse Design - Relational and
Dimensional Techniques,
Wiley Publishing, Inc., 2003

Efraim Turban, Ramesh Sharda, Dursun Delen,
David King, Janine E. Aronson
Business Intelligence: A Managerial Approach
Prentice Hall, 2010

Bibliography

SOURCES